Arab Open University - Egypt



Faculty of Artificial Intelligence

Information Technology and Computing Department

AI Personalized Nutrition
Recommendation System

Mario Fayez Sobhy Bebawe

23510186

TM471B: Final Year Project, Fall 2026/2027

Supervisor: Dr. Eid Emary

Declaration

I hereby certify that this material, which I now submit for assessment on the program of study leading to the award of bachelor of Information Technology and Computing in Computer Science is entirely my own work, that I have exercised reasonable care to ensure that the work is original, and does not to the best of my knowledge breach any law of copyright, and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work.

Student Name: Mario Fayez Sobhy Bebawe

Student ID: 23510186

Date: ____ / ____ / 2026

Acknowledgments

First and foremost, I would like to thank ALLAH for granting me the strength, patience, and ability to complete this project successfully.

I would like to express my sincere gratitude to my supervisor, Dr. Eid Emary, for his valuable guidance, continuous support, and constructive feedback throughout all stages of this project. His advice and encouragement were essential in helping me complete this work.

I would also like to thank the Faculty of Artificial Intelligence at the Arab Open University for providing the academic environment and resources necessary for completing this project.

Finally, I would like to express my deepest appreciation to my family for their unconditional support, encouragement, and patience throughout my academic journey.

Mario Fayez

Abstract

General diet plans ignore individual needs, and planning personal meals takes knowledge and time. This project built NutriPlan AI, a free web application that generated seven-day meal plans from USDA food data and 20 Egyptian dishes. It calculated energy targets, read preferences and allergies from free text using an AI language model with fixed safety rules, solved portions by optimization, and learned from the meals users ate or skipped. In tests, planned days had a mean calorie error of 2.3%, and no stated allergen reached a plan once it was understood. Testing on new wording exposed a weakness: after a safety fix, the parser blocked 68.8% of allergy items on sentences kept apart, so an allergy checklist was added. Simulated users received far fewer disliked foods with learning. No study with real users was completed.

Table of Contents

Declaration ..... **I**
Acknowledgments ..... **II**
Abstract ..... **III**
Table of Contents ..... **IV**
List of Figures ..... **VI**
List of Tables ..... **VII**
Abbreviations ..... **VIII**
Chapter 1 Introduction **1**
1.1 Problem definition 1
1.2 Significance of the Problem 1
1.3 Project Objectives 2
1.4 Scope of the Project 2
1.5 Project Methodology 2
1.6 Structure of the Report 3
Chapter 2 Literature Review **4**
2.1 Background 4
2.1.1 Energy and Nutrient Needs 4
2.1.2 Recommender Systems 4
2.1.3 Language Models for Reading Preferences 5
2.1.4 Meal Planning as an Optimization Problem 6
2.2 Related Projects 6
2.2.1 Project 1: MyFitnessPal 6
2.2.2 Project 2: Noom 7
2.2.3 Project 3: Eat This Much 7
2.2.4 Project 4: Cronometer 7
2.3 Related Research 8
2.4 Summary 9
Chapter 3 Requirements and Analysis **10**
3.1 Functional Requirements 10
3.2 Non-Functional Requirements 10
3.3 System Analysis 11
3.3.1 Software Requirements 11
3.3.2 Hardware Requirements 12
3.3.3 Other Requirements 12

3.4 System Modeling 12
3.4.1 Use Case Diagram 12
3.4.2 Data Flow Diagram 13
3.4.3 Entity-Relationship Diagram 14
3.5 Testing and Evaluation Plan 15
3.6 Summary 16
Chapter 4 Design, Implementation, and Testing **17**
4.1 Detailed Design 17
4.2 Implementation 19
4.2.1 Experimental Setup 20
4.2.2 Description of Datasets 20
4.2.3 User Interface 20
4.2.4 Application Components 28
4.3 Testing 29
4.4 Summary 30
Chapter 5 Results and Discussion **31**
5.1 Findings 31
5.1.1 Experimental Results 31
5.1.2 Ablation Study 34
5.2 Goals Achieved 35
5.3 Further Work 35
5.3.1 Enhancements and Future Features 35
5.3.2 Limitations and Incomplete Work 36
5.4 Social and Ethical Implications 36
5.5 Summary 37
Chapter 6 Conclusions **38**
6.1 Summary of the Work 38
6.2 Main Contributions 38
References **40**
الملخص العربي **43**

List of Figures

Figure 1-1 The project's eight phases 3
Figure 2-1 Main families of recommender systems 5
Figure 2-2 A model finds the foods; cue phrases and the allergen dictionary decide what to do 6
Figure 3-1 Use case diagram 13
Figure 3-2 Data flow diagram (level 1) 14
Figure 3-3 Entity-relationship diagram 15
Figure 4-1 System architecture 17
Figure 4-2 How free text becomes food rules 18
Figure 4-3 How each meal is planned 19
Figure 4-4 How each week teaches the next 19
Figure 4-5 Welcome page with sign-in 21
Figure 4-6 Profile page 22
Figure 4-7 Allergy checklist, preferences and consent 23
Figure 4-8 Plan page: targets, what was understood, and a request to check 24
Figure 4-9 Meal cards 25
Figure 4-10 Comparing the current meal with a new version 25
Figure 4-11 Day totals against targets 26
Figure 4-12 Nutrients page 26
Figure 4-13 Shopping list, in raw or dry amounts to buy 27
Figure 4-14 Progress page: finishing the week and tracking weight 27
Figure 4-15 Solving a meal's portions (src/planner.py) 28
Figure 4-16 Updating the taste vector (src/feedback.py) 29
Figure 4-17 Health wording overrules softer labels (src/parser.py) 29
Figure 5-1 A run of the automated test suite 31
Figure 5-2 Allergy items handled by the parser, on each set of sentences 33
Figure 5-3 Simulated testers with and without weekly learning 33
Figure 5-4 Days on target, with and without nutrient-aware planning (84 days) 34

List of Tables

Table 2-1 Comparison of related applications, from their public pages 7
Table 2-2 Comparison of related research 8
Table 3-1 Functional requirements 10
Table 3-2 Non-functional requirements 11
Table 3-3 Software used 11
Table 3-4 Hardware needs 12
Table 3-5 Evaluation plan 15
Table 4-1 Share of daily calories by goal 18
Table 4-2 Cleaning the food data 20
Table 4-3 Main components of the application 28
Table 4-4 Testing methods 30
Table 5-1 Planned days against targets (30 random users, 210 days) 32
Table 5-2 Allergy items blocked by the parser, by set of sentences 32
Table 5-3 Ablation study: what each part adds 34
Table 5-4 Objectives against results 35

Abbreviations

AI	Artificial Intelligence
BERT	Bidirectional Encoder Representations from Transformers
BMI	Body Mass Index
BMR	Basal Metabolic Rate
CSV	Comma-Separated Values
DASH	Dietary Approaches to Stop Hypertension
KNN	K-Nearest Neighbors
NER	Named Entity Recognition
PBKDF2	Password-Based Key Derivation Function 2
PDF	Portable Document Format
PIN	Personal Identification Number
SR Legacy	Standard Reference Legacy (USDA food data release)
TDEE	Total Daily Energy Expenditure
USDA	United States Department of Agriculture

Chapter 1
Introduction

Food choices shape long-term health. A global analysis estimated that 11 million deaths in 2017 were attributable to dietary risks, with high sodium and low intake of whole grains and fruit as the leading factors (Afshin et al., 2019). The World Health Organization (2026) describes unhealthy diets as a major risk factor for disease and disability, and links healthy diets to protection against diabetes, heart disease, stroke and cancer.

Knowing this does not make it easy to act on it. A person who wants to eat well has to work out how much energy they need, turn that into meals, respect their allergies and tastes, and repeat this every week. Mobile apps now help with parts of the job, but most of them either record what was eaten or hand out a general plan. Advances in natural language processing and recommender systems make it possible to do more: to read what a person types in their own words, and to learn from what they actually eat.

This project built NutriPlan AI, a working web application that creates a personalized seven-day meal plan from real food data, reads preferences and allergies from free text, and improves the following week's plan from the meals the user ate or skipped. It also includes Egyptian dishes, so the plans suit the people it was built for.

1.1 Problem definition

Three problems were addressed. First, planning is hard to do by hand. Unhealthy diets remain a major risk to health (World Health Organization, 2026), yet calculating a calorie target, dividing it into protein, carbohydrate and fat, and building meals that meet those numbers takes knowledge and time that most people do not have. Second, general plans ignore the individual. A plan written for an average adult does not account for a person's age, body size, activity level, goal, allergies or culture. Third, plans do not adapt. If a person keeps skipping a food, a fixed plan keeps serving it, and food recommendation research treats such changing, personal tastes as a central challenge (Trattner and Elsweiler, 2017).

Allergies make the problem more serious than convenience. A missed allergy can cause real harm, so a system that reads free text must be designed around the fact that a language model will sometimes be wrong (Niszczota and Rybicka, 2023).

1.2 Significance of the Problem

The scale of diet-related disease (Afshin et al., 2019) means that even a small improvement in everyday eating habits matters. A tool that removes the arithmetic and adapts to the person lowers the effort of eating well.

The problem also matters technically. Food is a difficult area for recommender systems, because a good suggestion must be liked by the user and also meet nutritional targets (Trattner and Elsweiler, 2017). Safety adds a further constraint that ordinary recommenders do not face: a film recommendation can be wrong without consequence, but a meal recommendation cannot. A study of 56 diets written by ChatGPT for hypothetical people with food allergies found it generally accurate, but able to produce harmful diets (Niszczota and Rybicka, 2023). This project shows one way of combining language models with fixed rules so that safety does not depend on the model alone. It also reports where that approach still falls short.

1.3 Project Objectives

The objectives of the project were to:

1. Calculate each user's daily calorie, macronutrient and key nutrient targets from their profile and goal.
2. Prepare a clean food dataset from a reliable public source, with allergen and diet information for every food.
3. Read preferences and allergies from free text, and apply allergies as hard rules.
4. Generate a seven-day meal plan automatically, with portions that meet the user's targets.
5. Improve each following week's plan from the meals the user ate, skipped or replaced.
6. Provide a simple web interface, with accounts that stay private to each user.
7. Evaluate accuracy, safety and learning with measurable tests, and state the limits of those tests.

1.4 Scope of the Project

Within scope: adults who want general guidance on eating; calorie, macronutrient and nutrient targets; meals built from the USDA SR Legacy foods plus 20 Egyptian dishes; free-text preferences in English; weekly learning; private accounts; shopping lists, printable plans and weight tracking; and two optional diet modes, diabetes-friendly and low-salt, offered as general guidance.

Outside scope: medical diagnosis or treatment; plans for children, pregnant women or people with eating disorders; food allergy testing; the exact ingredients of packaged or restaurant foods; and reading preferences written in Arabic. Allergens are identified from food names, so an ingredient not named in a food's description cannot be detected. The application tells users that it is for education only and is not medical advice.

1.5 Project Methodology

The work followed eight phases, shown in Fig. 1-1, each built, tested and reviewed before the next one started. An iterative approach suited the project,

because the results of one phase often changed the next. The planner, for example, first met calorie targets but missed calcium, which led to nutrient-aware planning. Testing on new sentences late in the project exposed an allergy weakness in the parser, which led to a dedicated safety layer.

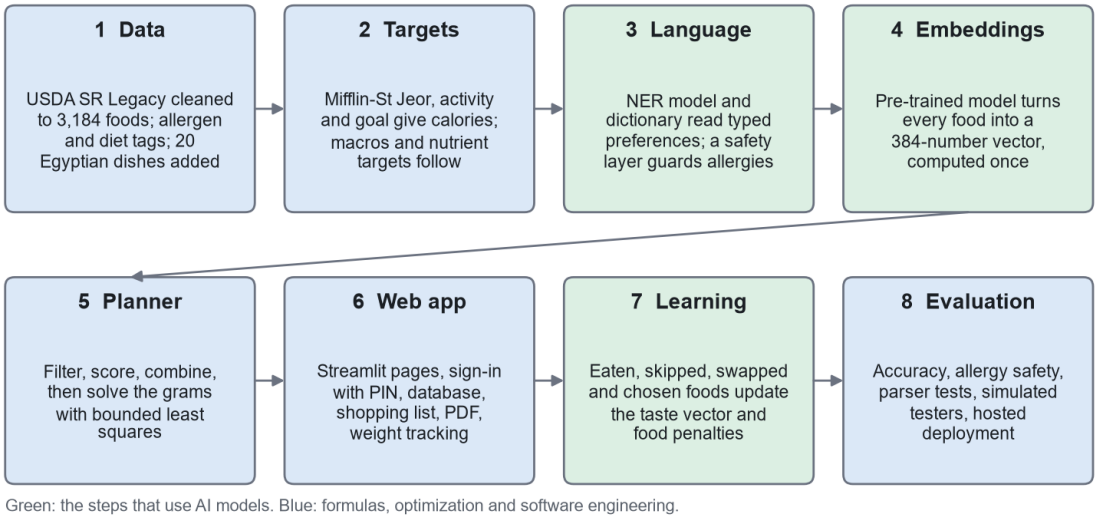


Figure 1-1 The project's eight phases

Two decisions changed from the first part of this report. A K-Nearest Neighbors search over nutrient values was planned, but a food close to a meal's numbers is not necessarily a sensible part of a meal, so meals are instead assembled from food roles and their portions are solved by optimization. Nearest-neighbor search is still used, in the space of food embeddings, for substitutions. Flask was also replaced by Streamlit, which let the work focus on the recommendation logic. The proposal also listed a trained recommendation model, and none was trained. A single new user has no history to train on, so the project uses pre-trained language models with arithmetic on their vectors, which also keeps it free to run on a normal processor.

1.6 Structure of the Report

Chapter 2 presents the background to the project and reviews related applications and research. Chapter 3 focuses on the requirements and analysis of the system, including its models and the evaluation plan. Chapter 4 describes the design, implementation and testing of the application. Chapter 5 discusses the results, the goals achieved, further work, and the social and ethical implications. Finally, Chapter 6 concludes the report by summarizing the work and its main contributions.

Chapter 2
Literature Review

This chapter explains the knowledge the project builds on, then reviews existing applications and research. For each one it states what is missing and how this project addresses part of that gap. The aim is not to claim that the project replaces these works, several of which have far more users and data, but to place it among them accurately.

2.1 Background

A personalized meal planner draws on four areas: nutrition science, which sets the targets; recommender systems, which decide what a person is likely to accept; language models, which read what the person types; and optimization, which turns foods into portions.

2.1.1 Energy and Nutrient Needs

Daily energy needs are usually estimated in two steps. The basal metabolic rate (BMR) is the energy used at rest, and it is multiplied by a factor for activity to give the total daily energy expenditure (TDEE). This project uses the Mifflin-St Jeor equation (Mifflin et al., 1990), which needs weight, height, age and biological sex. A systematic review found that it predicted resting metabolic rate within 10% of the measured value in more non-obese and obese people than the other equations reviewed, while warning of noteworthy errors for individuals (Frankenfield et al., 2005). An energy target is therefore a sensible starting estimate, not a measurement.

The energy target is then adjusted for the person's goal and divided into macronutrients: protein and carbohydrate provide about 4 kcal per gram, and fat about 9 kcal per gram. A common rule of thumb is that a kilogram of body weight corresponds to about 7,700 kcal, which Hall (2008) shows is an approximation, because the energy content of weight lost depends on the person's body fat. Any forecast of weight change made from a calorie target should be read the same way.

Calories alone do not make a healthy plan. Public guidance also sets values for individual nutrients, such as limiting saturated fat to under 10% of energy (U.S. Department of Agriculture and U.S. Department of Health and Human Services, 2020) and sodium to 2,300 mg a day to reduce the risk of chronic disease (National Academies of Sciences, Engineering, and Medicine, 2019). Reference values for iron and calcium come from the Institute of Medicine (2001, 2011). The World Health Organization (2026) advises keeping free sugars below 10% of energy. For conditions such as high blood pressure, dietary patterns have been tested directly: the DASH diet lowered blood pressure in a controlled trial (Appel et al., 1997). These values gave the project more to aim for than calories and macronutrients.

2.1.2 Recommender Systems

Recommender systems predict which items a person will value. They are

commonly divided into content-based methods, which compare the properties of items with what this user liked (Lops et al., 2011); collaborative filtering, which uses the behavior of many similar users; and hybrids, which combine methods to cover each other's weaknesses (Burke, 2002), as shown in Fig. 2-1. Collaborative filtering needs a large community of users before it works well, which a new application does not have (Adomavicius and Tuzhilin, 2005). A content-based approach, in which foods are compared with those the same person accepted, can work from a single user's first week.

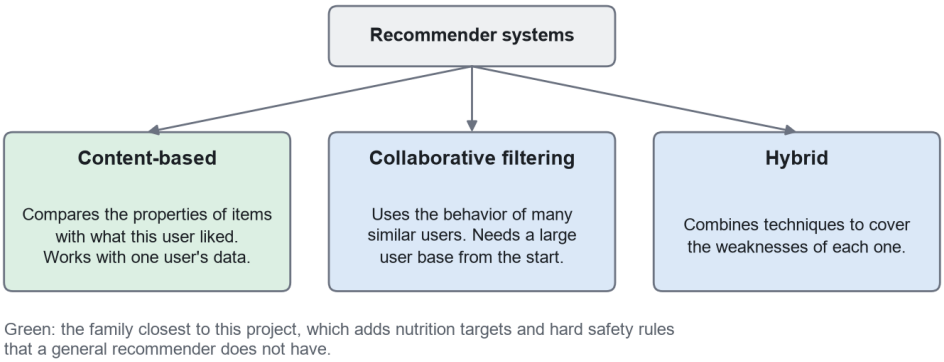


Figure 2-1 Main families of recommender systems

Food is a harder domain than most. People's tastes interact with health, culture, cost and routine. Most existing work suggests items to individual users based on their preferences or health problems (Tran et al., 2018). A survey of the field reports that standard approaches work poorly for constrained needs such as allergies and vegan diets, and that constraint-based approaches are found surprisingly rarely in the literature (Trattner and Elsweiler, 2017). This finding shaped the design of this project: allergies and diets are treated as hard rules applied before any recommendation is made.

2.1.3 Language Models for Reading Preferences

Letting people type freely is easier for them than filling in long forms, but it requires the system to understand text. Named Entity Recognition (NER) is the task of finding and labeling words that refer to something specific, such as a food. BERT (Devlin et al., 2019) is a neural language model that represents each word using the words around it, and it can be fine-tuned for a task like this with little labeled data.

FoodBase is a corpus of 1,000 recipes with 12,844 manually checked food annotations (Popovski et al., 2019). Models fine-tuned on it reached a macro F1 score of up to 94.31% at telling food from non-food, although the authors warn of weaknesses on texts of a different nature from the recipes they were trained on (Stojanov et al., 2021). The FoodBaseBERT-NER model used in this project is a BERT model fine-tuned on FoodBase, and its public description gives no evaluation figures (Dizex, no date). As Fig. 2-2 shows, such a model finds which words are foods, but it does not say how the person feels about them. That second step needs rules of its own, and as Chapter 5 shows, this is where most of the difficulty lies.

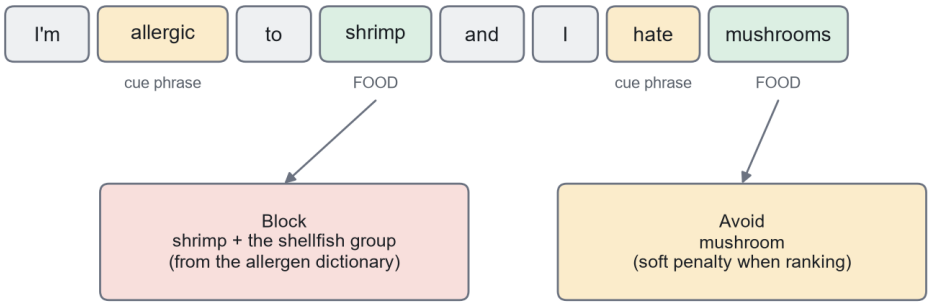


Figure 2-2 A model finds the foods; cue phrases and the allergen dictionary decide what to do

Sentence embeddings turn a piece of text into a list of numbers, so that texts with similar meanings end up close together (Reimers and Gurevych, 2019). The all-MiniLM-L6-v2 model produces 384 numbers for each short text. Comparing foods in this space allows the system to recognize that grilled fish resembles grilled chicken more than it resembles a chocolate bar, even when their calories match. A running average of the vectors of foods a person eats forms a simple taste profile, and it needs no model training of its own.

2.1.4 Meal Planning as an Optimization Problem

Choosing foods to meet nutritional targets is one of the oldest optimization problems; it was studied by Stigler (1945) as the cost of meeting nutritional needs. A purely numeric solution, however, can produce meals nobody would eat. This project keeps the numeric part small: once sensible foods have been chosen for a meal, only their portions are optimized, using bounded-variable least squares (Stark and Parker, 1995), available in the SciPy library (Virtanen et al., 2020). The bounds keep every portion realistic.

2.2 Related Projects

Four widely used applications were reviewed from their public pages on 2 November 2026, as summarized in Table 2-1. This is a review of what the companies state, not a hands-on test, so "not described" means only that the pages reviewed do not mention a feature, and paid plans may offer more.

2.2.1 Project 1: MyFitnessPal

MyFitnessPal (2026) is mainly a tracker. It states that it has over 18 million foods in its database, and offers logging by search, photo and voice, with tools such as barcode scanning and custom macro goals in its paid plan. Its strength is the size of its data. It lists a meal planner among its screens but gives no detail of it. **What is missing:** the person still does most of the deciding. **How this project helps:** it starts from a complete plan that meets the targets, so the person reacts to suggestions instead of searching for food.

2.2.2 Project 2: Noom

Noom (2026) combines food logging, optional coaching, meal plans and recipes, and now also access to medication for eligible users. Its focus is on changing behavior, with human support. Its plans are paid, starting at about $17 a month on an annual plan. **What is missing:** its main features are not free, which limits who can use them, and no detail is given of how meals adapt to a person's skipped foods. **How this project helps:** it is free to use and runs on free tools, and its learning step is described and measured openly in this report.

2.2.3 Project 3: Eat This Much

Eat This Much (2026) is the closest to this project. It generates meal plans from a calorie target and number of meals, supports diet types such as vegetarian and keto, and keeps a grocery list that updates as meals change. Its meal planner page also allows allergies to be entered. App store descriptions limit the free version to a day's plan, with weekly plans in the paid version. **What is missing:** preferences are entered by selection, and learning from skipped meals is not described. **How this project helps:** a person can type in their own words, a full week is free, and each week's plan learns from the previous one. It also includes Egyptian dishes.

2.2.4 Project 4: Cronometer

Cronometer (2026) stands out for detail. It tracks up to 95 nutrients and compounds using more than a million verified foods, and its target wizard helps set goals. Its users are often people who want accuracy. **What is missing:** its homepage does not describe generating meal plans, so the user still has to plan. **How this project helps:** nutrients are used when planning instead of only being reported afterwards, although it tracks far fewer of them (seven besides calories and macronutrients).

Table 2-1 Comparison of related applications, from their public pages

Feature	MyFitnessPal	Noom	Eat This Much	Cronometer	This project
Main purpose	Tracking	Coaching and behavior	Meal planning	Detailed tracking	Meal planning
Plans made for the user	Not described	Meal plans listed	Yes	Not described	Yes, 7 days
Allergy input	Not described	Not described	By selection	Not described	Free text and rules
Learns from skipped meals	Not described	Not described	Not described	Not described	Yes
Egyptian dishes	Not described	Not described	Not described	Not described	20 dishes
Cost	Free and paid	Paid	Free and paid	Free and paid	Free
Size of food data	Over 18 million	Over 1 million	Not checked	Over 1 million	3,204 foods

The comparison also shows what this project lacks. Its food table is thousands of times smaller than the largest, it has no barcode scanner or mobile app, and it has no human coaching.

2.3 Related Research

Four studies closest to this work are reviewed below, and Table 2-2 compares them.

Yang et al. (2017), Yum-me. This system learns tastes through a visual quiz of food images and then recommends from about 50,000 recipes that fit a person's diet type and nutritional goals. A field study with 227 anonymous users supported its learning method, and a study with 60 people found a 42.63% improvement in the recommendation acceptance rate. This evidence from real people is stronger than anything this project has. **What is missing:** it recommends individual dishes rather than building a week with portions, and restrictions are chosen from a survey. **How this project helps:** it builds complete days with portions in grams, and reads restrictions from free text.

Elsweiler and Harvey (2015). This work tested whether daily meal plans could be generated automatically for a sample of 100 user profiles combined with food preference data from 64 people, treating the balance between what users want and what is nutritionally appropriate as something to optimize. **What is missing:** it relies on recipe ratings, which a new user does not have. **How this project helps:** it learns from a single user's eaten and skipped meals, with no ratings from other people.

Khamesian et al. (2025), NutriGen. This framework asks large language models for meal plans, with USDA data as a reference. On ten simulated profiles, the best model missed calorie targets by 1.55% (34 kcal). The authors report incomplete plans caused by output limits, inconsistent calorie estimates, and a lack of user control, and they note that simulated data limits how far the results apply. **What is missing:** correct arithmetic depends on a model that can be wrong, and the evaluation does not cover allergy safety. **How this project helps:** portions are calculated, so totals are exact for the data used, and models are used only to read text and compare foods. The two studies used different tests, so their percentages cannot be compared directly.

Niszczota and Rybicka (2023). This study examined 56 diets written by ChatGPT for hypothetical people with food allergies, and found it generally accurate but capable of producing harmful diets, with errors in portions and calories being the most common. **What is missing:** it evaluates a general model rather than proposing a safer design. **How this project helps:** allergies are applied through a fixed allergen dictionary, with the model only proposing which words are foods. Chapter 5 tests how far this goes, and it is not a complete solution.

Table 2-2 Comparison of related research

Work	Method	Evidence	Gap addressed by this project
Yang et al. (2017)	Image quiz, online learning	227 and 60 real users	Whole week with portions; free-text restrictions
Elsweiler and Harvey (2015)	Ratings with optimization	100 profiles, 64 people's preferences	No ratings from other people needed
Khamesian et al. (2025)	Large language models	10 simulated profiles	Calculated portions; allergy rules
Niszczota and Rybicka (2023)	Evaluation of ChatGPT	56 diets, hypothetical people	A design with fixed safety rules
This project	NER, embeddings, least squares, rules	Simulated testers; no real-user study yet	See Chapter 5

Three lessons from these studies shaped the design of this project. First, language models are useful for reading text but should not be trusted with arithmetic or safety, as NutriGen's inconsistent calorie estimates and the harmful ChatGPT diets show. Portions are therefore calculated, and allergies are checked against a fixed dictionary. Second, a model's published score does not transfer automatically. The food NER models were trained on recipes, and their authors warn about other kinds of text; a person describing their allergies writes quite differently from a recipe. This warning turned out to be accurate, and it is why this project tested its parser on new wording. Third, the strongest evidence comes from real people, as in Yum-me. Simulated users, used by NutriGen and by this project, can show that a system behaves as designed, but not that people would accept it.

2.4 Summary

The review showed that commercial applications lean towards tracking or paid coaching, while research systems tend to focus on one piece of the problem: learning tastes, generating plans, or reading text. Constraint handling for allergies is rarely studied, even though it is the part where errors can do harm. The gap this project addresses is a free system that combines free-text preferences, fixed allergy rules, calculated portions, weekly learning from one person's behavior, and local dishes in a single working application. Its clearest weakness compared with this work is evidence: it has not yet been tested with real users.

Chapter 3
Requirements and Analysis

This chapter defines what the system must do and how well it must do it, the resources it needs, and how it was modeled and evaluated. The requirements from the first part of this report were kept, and new ones were added as the project developed, such as private accounts and diet modes. The social and ethical issues that were in this chapter in the first part have moved to Section 5.4, as the template requires.

3.1 Functional Requirements

Functional requirements define what the system should do. Table 3-1 lists them, in order from creating an account to the weekly cycle. Those from the first part of the report are refined here; the others were added during development.

Table 3-1 Functional requirements

ID	The system shall
FR1	Let a user create a private account with a name and a 4 to 8 digit PIN, sign in and out, agree to how their data is used, and delete the account.
FR2	Collect age, sex, weight, height, activity level, goal and meals per day, and let the user change them.
FR3	Calculate daily calorie, protein, carbohydrate and fat targets, and reference values for six other nutrients.
FR4	Provide an allergy checklist, read preferences and other allergies from free text, and show what it understood.
FR5	Never include a food that breaks a stated allergy or diet rule that it has understood.
FR6	Ask the user when it cannot tell what a sentence about food or health means.
FR7	Generate a seven-day plan with three, four or five meals a day and portions in grams.
FR8	Offer Egyptian dishes often, sometimes or never, and two optional diet modes (diabetes-friendly, low-salt).
FR9	Let the user mark each meal as eaten, and save this.
FR10	Let the user swap one food, or compare a new version of a meal or day with the current one and choose.
FR11	Improve the following week's plan from eaten, skipped, swapped and chosen foods.
FR12	Produce a shopping list, and downloads of the plan as PDF and CSV.
FR13	Record weight over time, show progress, and update targets when the weight changes.

An administrator role with a dashboard was considered in the first part of the report but was dropped. No screen in the system lets one person view another person's profile, which suits the private nature of health data.

3.2 Non-Functional Requirements

Non-functional requirements describe the qualities of the system. They are stated with measures where possible, so that Chapter 5 can check them (Table 3-2). The measures were set during development, not before it started.

Table 3-2 Non-functional requirements

ID	Quality	Requirement
NFR1	Accuracy	Planned days stay close to targets: mean calorie error under 5%, and at least 90% of days within 10%.
NFR2	Safety	No food with a stated, understood allergen is served. Allergy rules do not depend on a machine learning model alone.
NFR3	Performance	A weekly plan is ready within 10 seconds, without a graphics card.
NFR4	Security	PINs are stored only as salted hashes, and repeated wrong PINs lock the account for a time.
NFR5	Privacy	Names cannot be discovered from the sign-in form, and an address alone cannot open a profile.
NFR6	Usability	Every feature is one click from the menu, in plain language, on a computer or phone screen.
NFR7	Reliability	Accounts and plans survive a server restart on the hosted version.
NFR8	Portability	Runs on Windows and Linux, on a PC or in a container, with the same database code.
NFR9	Cost	Uses only free data, free models and free tools, with no paid programming interfaces.
NFR10	Maintainability	Automated tests cover each feature, and can be run with one command.

3.3 System Analysis

The system takes a small amount of structured data and a sentence of free text, and produces a week of meals. Its main analysis decision was to separate the parts that need AI from those that do not. Calorie targets are formulas, and filtering foods is a rule, so neither needs a model; using one would only add error. Reading free text, comparing foods by meaning, and learning tastes are the parts that benefit from AI models. A further decision was that models must run on the server with no paid service, so that the application stays free and no one's health data is sent to a third party for processing.

3.3.1 Software Requirements

Table 3-3 lists the main software. Python was chosen for its machine learning libraries. Streamlit replaced the Flask framework planned earlier, because it allowed a complete interface to be written in Python alone. SQLite needs no setup on a PC, while PostgreSQL keeps data safe on a hosted service whose servers have only temporary disks.

Table 3-3 Software used

Software	Version	Purpose
Python	3.13	Programming language
Streamlit	1.65	Web interface
pandas, NumPy, SciPy	3.0, 2.5, 1.18	Data handling and portion solving
Transformers, PyTorch	5.19, 2.14	Running the NER model
Sentence Transformers	6.1	Food embeddings
SQLite, PostgreSQL (Neon)	Latest available	Database on a PC, and on the hosted site
fpdf2	2.8	PDF plans
pytest	Latest available	Automated tests
Docker, Vercel	Latest available	Packaging and free hosting
Git, GitHub, Visual Studio Code	Latest available	Version control and development

3.3.2 Hardware Requirements

The system needs no special hardware, as the models are small and run on a normal processor. Table 3-4 gives the figures measured in a container limited to one processor and 2 GB of memory, which matches the free hosting plan. The memory peak was 0.46 GB for a plan without preference text. With preference text it was 0.96 GB, measured on a Windows PC.

Table 3-4 Hardware needs

Item	Measured or needed
Processor	One processor core is enough; no graphics card
Memory	2 GB is enough for the server; peak measured under 1 GB
Storage	3.52 GB for the container image, including both models
User's device	Any computer or phone with a modern web browser

3.3.3 Other Requirements

The USDA SR Legacy dataset and the two pre-trained models must be downloaded once. After that the application runs without contacting any outside service, apart from the hosted database and the web fonts. Users need an internet connection to reach the hosted site, and a hosting account and a free database are needed to publish it.

3.4 System Modeling

Three standard diagrams were used, each answering a different question: what users can do, how data moves, and how data is stored. They replace the diagrams in the first part of the report, which showed the planned KNN design, and they were redrawn from the finished code.

3.4.1 Use Case Diagram

Fig. 3-1 shows the functions available to the only actor, the user. A use case diagram was chosen because it shows the scope of the system without any technical detail. The green use cases are not started directly. They are included by another use case: generating a plan always includes calculating targets, reading preferences and building meals.

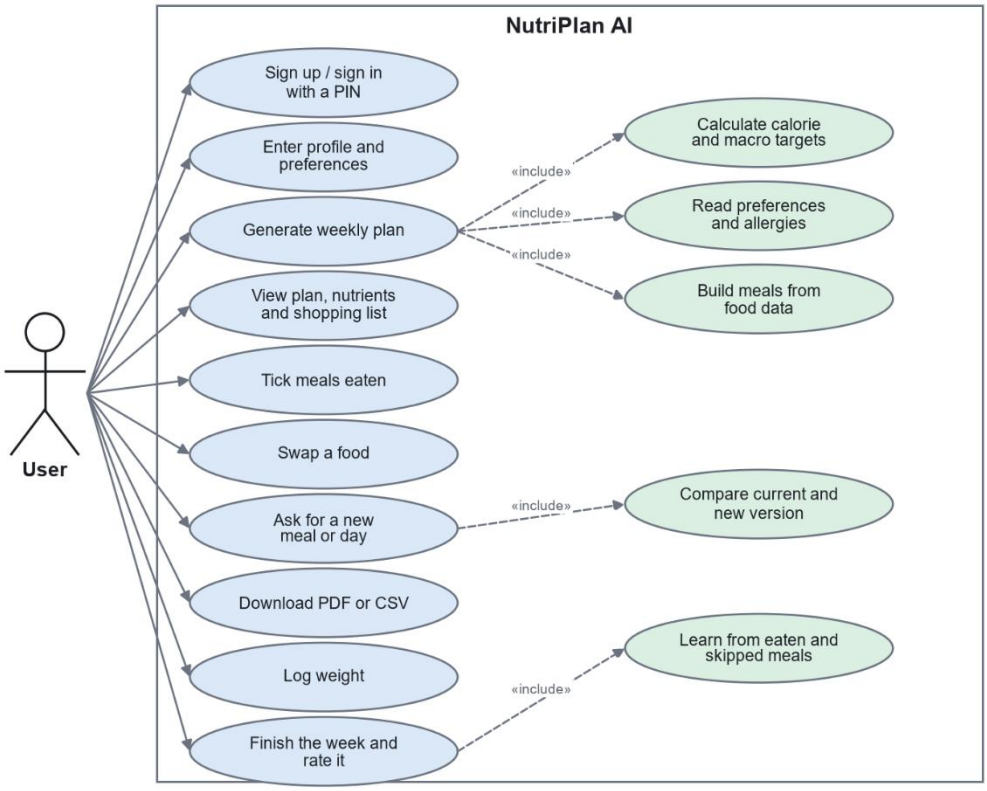


Figure 3-1 Use case diagram

3.4.2 Data Flow Diagram

Fig. 3-2 shows how data moves between the user, the six main processes and the data stores. It was chosen because the core of this system is a flow: a profile and text become targets and rules, which become a plan, whose use becomes learning for the next plan. The circles are processes, and the open rectangles are data stores. The learning process (6.0) feeds back into planning (4.0), which is what makes the system adaptive.

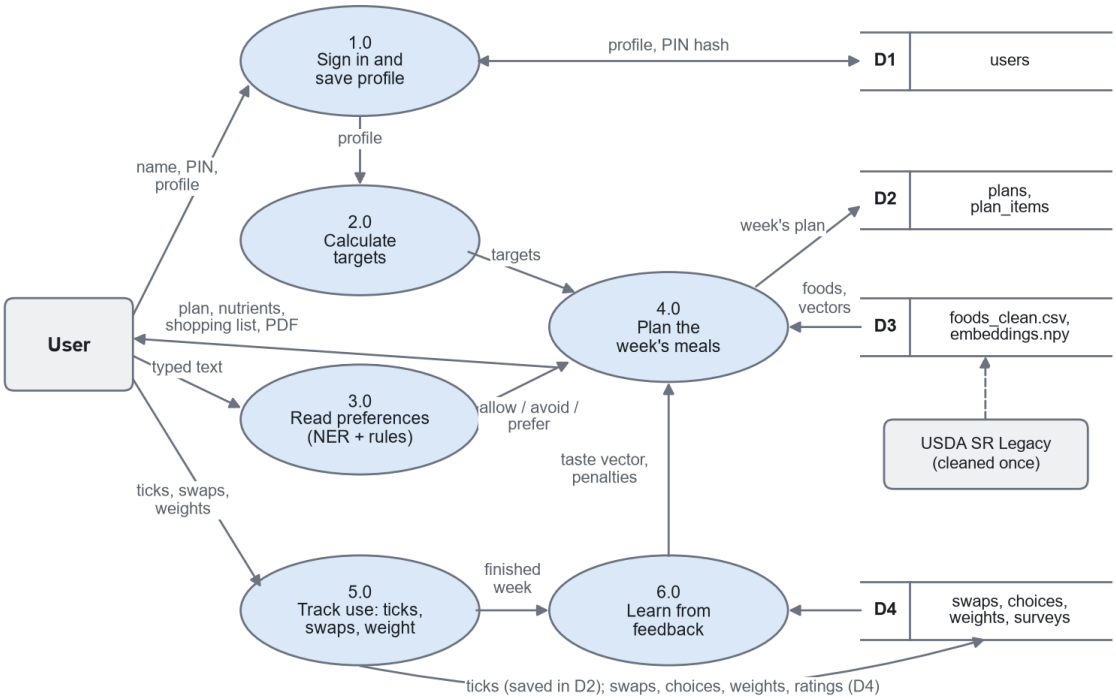


Figure 3-2 Data flow diagram (level 1)

3.4.3 Entity-Relationship Diagram

Fig. 3-3 shows the seven tables of the database. A user has many plans and many weight entries, and a plan has many food items, swaps and choices, and at most one survey. Eaten or skipped is stored on each plan item. The food table is kept as a read-only file because it never changes while the application runs, so plan items refer to it by food ID and also keep their own copy of the name and nutrients, which means a saved plan stays correct if the data is rebuilt.

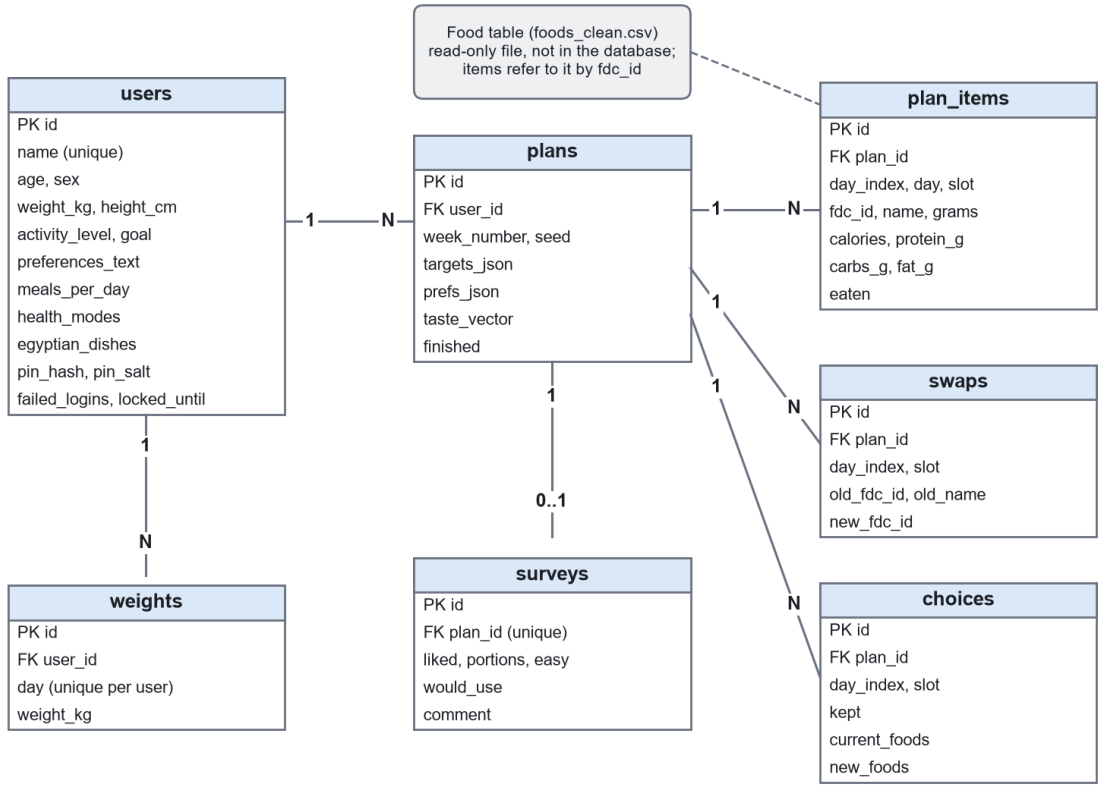


Figure 3-3 Entity-relationship diagram

3.5 Testing and Evaluation Plan

Testing was planned at two levels. Automated tests check that each part works as intended, from single formulas to complete flows through the interface. Evaluation then measures how well the whole system meets its goals, using the metrics in Table 3-5.

Table 3-5 Evaluation plan

Question	Method	Metric
Do plans meet targets?	Random user profiles, 7 days each	Mean % error; days within 10%
Are allergies respected?	Users with stated allergies	Food items containing an allergen
Is the allergen dictionary right?	Hand-labeled foods	Precision and recall
Does the parser understand people?	Sentences on new wording	Allergy recall; label recall
What does each part add?	Ablation: remove one part at a time	Change in the scores above
Does the system learn?	Simulated users with hidden dislikes	Meals eaten, with and without learning
Are nutrients covered?	Nutrient-aware against macros only	Days on target; days over a safe limit
Is it fast enough?	Timing of plan generation	Mean and maximum seconds

The first part of this report also planned to measure user satisfaction. The tools for this were built: a rating form in the application, a sheet for volunteers' test sentences, and a summary of use without names. A study with real people was not completed, so simulated users stand in for it where possible. Satisfaction cannot be simulated, so it is

reported as not measured (Chapter 5).

3.6 Summary

This chapter set out 13 functional and 10 non-functional requirements, with measures that Chapter 5 checks. It explained the software and hardware needed, showing that the system runs on modest, free resources, and it modeled the system with use case, data flow and entity-relationship diagrams based on the finished code. It closed with the plan for testing and evaluation, including the part that was not completed.

Chapter 4
Design, Implementation, and Testing

This chapter describes how the system was designed, built and tested. The result is a working web application, available to run on a PC and hosted online. Its design combines a layered structure with a hybrid of AI models and fixed rules.

A hybrid was chosen over two alternatives. A purely rule-based system was rejected because rules alone cannot recognize the open-ended ways people describe food or learn what they like. A purely AI approach, such as asking a large language model for a whole plan, was rejected because the numbers and the safety of a plan must be dependable, and such models have been shown to make errors in both (Niszczota and Rybicka, 2023; Khamesian et al., 2025). In this design, each technique does the job it is best at.

4.1 Detailed Design

The overall architecture is illustrated in Fig. 4-1. First, the browser shows pages built by the Streamlit interface. Second, the interface calls application logic, written as separate modules with no knowledge of the screens, which allows each one to be tested alone. Third, the logic uses the models and read-only data files, and saves each person's information through one database module.

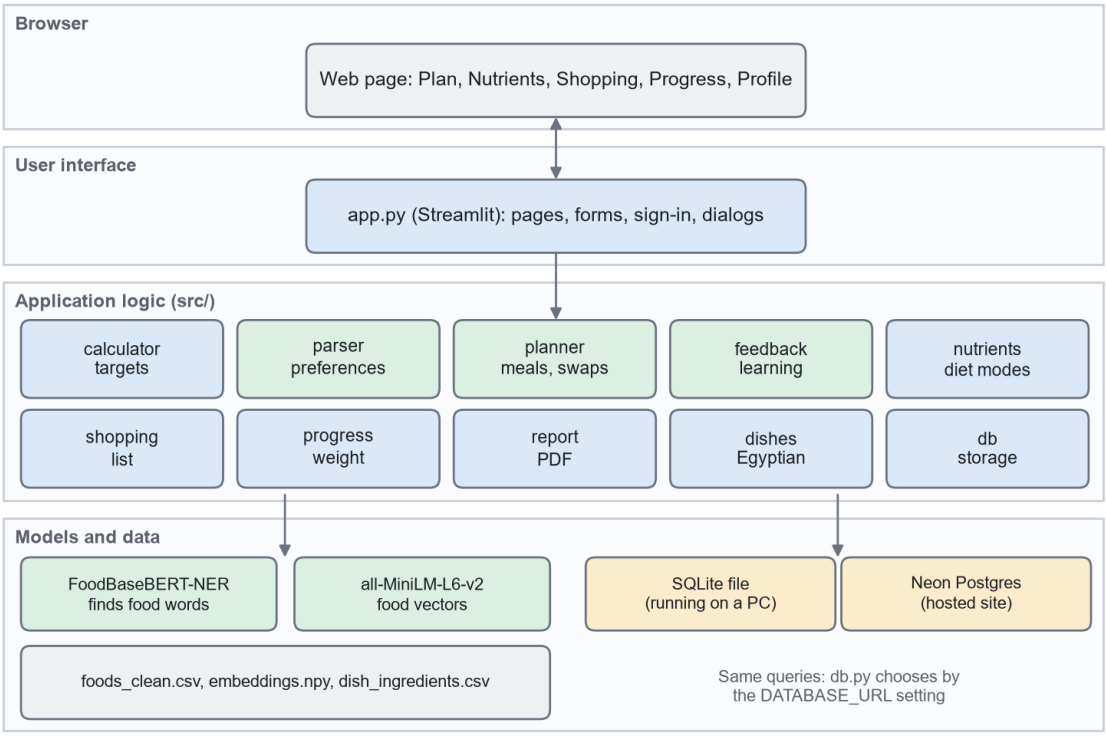


Figure 4-1 System architecture

Targets. Basal metabolic rate uses the Mifflin-St Jeor equation: 10 x weight (kg)

+ 6.25 x height (cm) - 5 x age, plus 5 for men or minus 161 for women. It is multiplied by 1.2, 1.55 or 1.725 for activity, then adjusted by -20% to lose weight or +10% to gain muscle, with a floor of 1,500 kcal for men and 1,200 kcal for women. Calories are divided into macronutrients by goal (Table 4-1), and protein is capped at 2.2 g per kg of body weight, with the spare calories going to carbohydrate. For the 68 kg, 165 cm, 27-year-old test account in Fig. 4-8, the maintenance target is 2,451 kcal, and the protein cap reduces protein from 153 g to 150 g.

Table 4-1 Share of daily calories by goal

Goal	Protein	Carbohydrate	Fat	Calorie adjustment
Maintain weight	25%	45%	30%	None
Lose weight	35%	35%	30%	-20%
Gain muscle	30%	45%	25%	+10%

Allergy checklist. Everyone answers a checklist of ten allergen groups, or states that they have no food allergies (Fig. 4-7). A ticked group is always blocked and involves no language model, so safety for these groups does not depend on how a sentence is worded. Accounts from before this feature answer it at their next sign-in, and a ticked allergy rebuilds their plan.

Reading preferences. Free text remains for likes, dislikes and allergies that are not on the list. The flow in Fig. 4-2 starts by correcting spelling, using the food names and the NER model's own vocabulary so that ordinary words are left alone. Foods are then found by the model together with a dictionary scan, and each is labeled block, avoid or prefer by the nearest cue phrase. Health wording overrules softer labels: a body reaction, a condition such as celiac disease, or a doctor's advice counts as a block. Every blocked word is then checked against the allergen dictionary, which needs no machine learning. When the parser cannot tell what was meant, the application asks instead of staying silent. Over-blocking is accepted, because it is the safe direction.

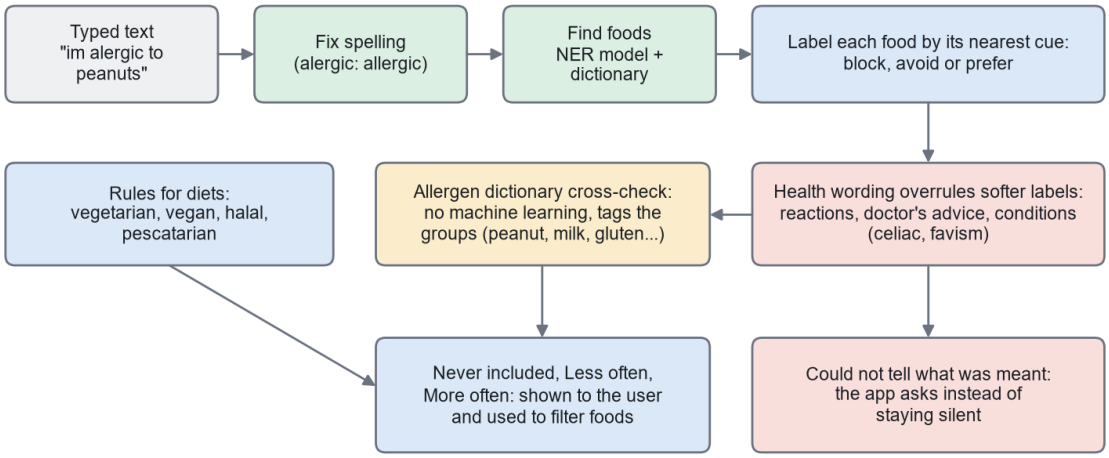


Figure 4-2 How free text becomes food rules

Planning meals. SR Legacy foods are ingredients, so each meal is assembled

from parts: protein, carbohydrate, vegetable and optional oil for lunch and dinner; a main food, carbohydrate and fruit for breakfast; or one Egyptian dish with optional sides. As Fig. 4-3 shows, candidates for each part are ranked by this score:

score = fit + taste similarity + stated preference + nutrient score - repeat penalties - learned penalties

The best five for each part are combined, and the grams of every combination are solved. Each meal aims at what is left of the day's target, so the last meal corrects earlier small errors. The foods the user cannot eat were already removed before this step, so no later score can bring them back.

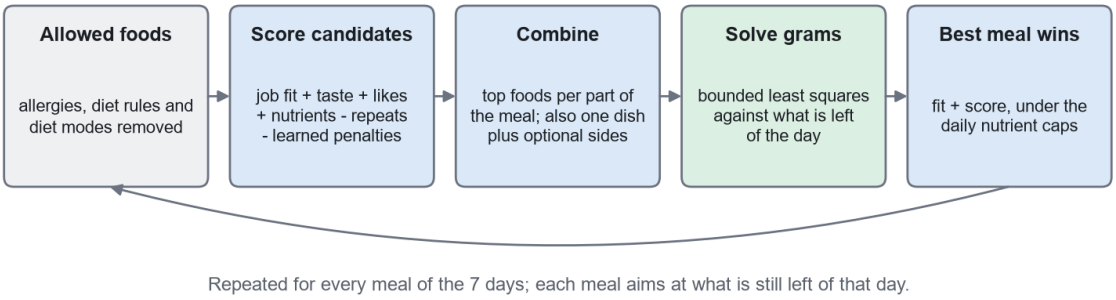


Figure 4-3 How each meal is planned

Learning. Fig. 4-4 shows the weekly loop. Eaten foods pull a 384-number taste vector towards them, and skipped foods push it away, which also affects similar foods never served. Exact foods and whole families, such as all beef foods, also gain penalties, and older weeks count for less. A swap counts as a skip, and a choice between two versions counts as half.

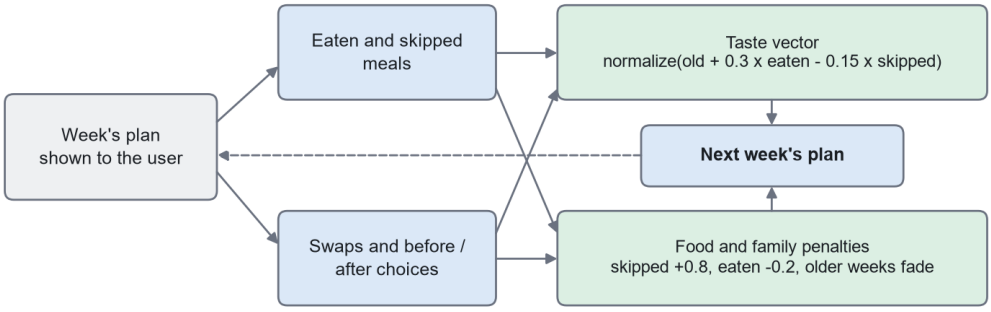


Figure 4-4 How each week teaches the next

4.2 Implementation

The system was written in Python as 19 modules plus the interface, and version control kept every step.

4.2.1 Experimental Setup

Development and evaluation ran on a Windows 11 laptop with an Intel Core i7-10750H processor and 16 GB of memory, using the processor only. Evaluation scripts use fixed random seeds, so results can be reproduced, and they save their results in data/evaluation. Simulated testers ran on a throwaway database, never the real one. The hosted version runs in a Docker container on Vercel's free plan in Frankfurt, with a free Neon PostgreSQL database.

4.2.2 Description of Datasets

Three data sources feed the system: 1) USDA SR Legacy, 2) a set of Egyptian dishes written for this project, and 3) the FoodBase corpus, used indirectly through the pre-trained NER model.

USDA SR Legacy (U.S. Department of Agriculture, Agricultural Research Service, 2018) is the final release of a public database of generic foods, with 7,793 foods in separate linked tables. These were joined into one row per food, and cleaned down to foods that make sense in a daily meal (Table 4-2). Each food received nutrient values per 100 g, a household portion, a role in a meal, and allergen and diet tags. Allergens are tagged by whole-word matching on the food's name, with exceptions for traps such as eggplant and coconut milk. Missing nutrient values were estimated from similar foods and marked: sugars were the most affected, at 24.1% of foods.

Table 4-2 Cleaning the food data

Step	Foods removed
Starting foods	7,793 (start)
Categories not used for meals (such as baby foods and fast food)	1,325
Processed forms, organ meats, game, foods with added salt	1,813
Raw or dry versions of foods eaten cooked	644
Brand-name products	549
Other (zero calories, flours, sauces, drinks, dry mixes, inconsistent values)	278
Foods kept	3,184
Egyptian dishes added	20, giving 3,204

Egyptian dishes. Because USDA has few Egyptian foods, 20 dishes were written as recipes of USDA ingredients, including ful medames, koshari, molokhia and mahshi. Each dish's nutrients and allergens are calculated from its ingredients, so koshari is tagged as containing wheat. These are home-style estimates, not laboratory measurements.

FoodBase (Popovski et al., 2019) trained the NER model, which this project did not retrain. For testing the parser, 32 sentences were written during development and three further sets of 141 sentences afterwards (Section 4.3).

4.2.3 User Interface

The interface has a top menu with five pages and works in light and dark themes. A review of user complaints about similar applications shaped its design: features are

one click from the menu, nutrients are in plain view, and status words stay neutral, such as "a little under or over" and never "bad". Figures 4-5 to 4-14 are screenshots of the real application, using a test account.

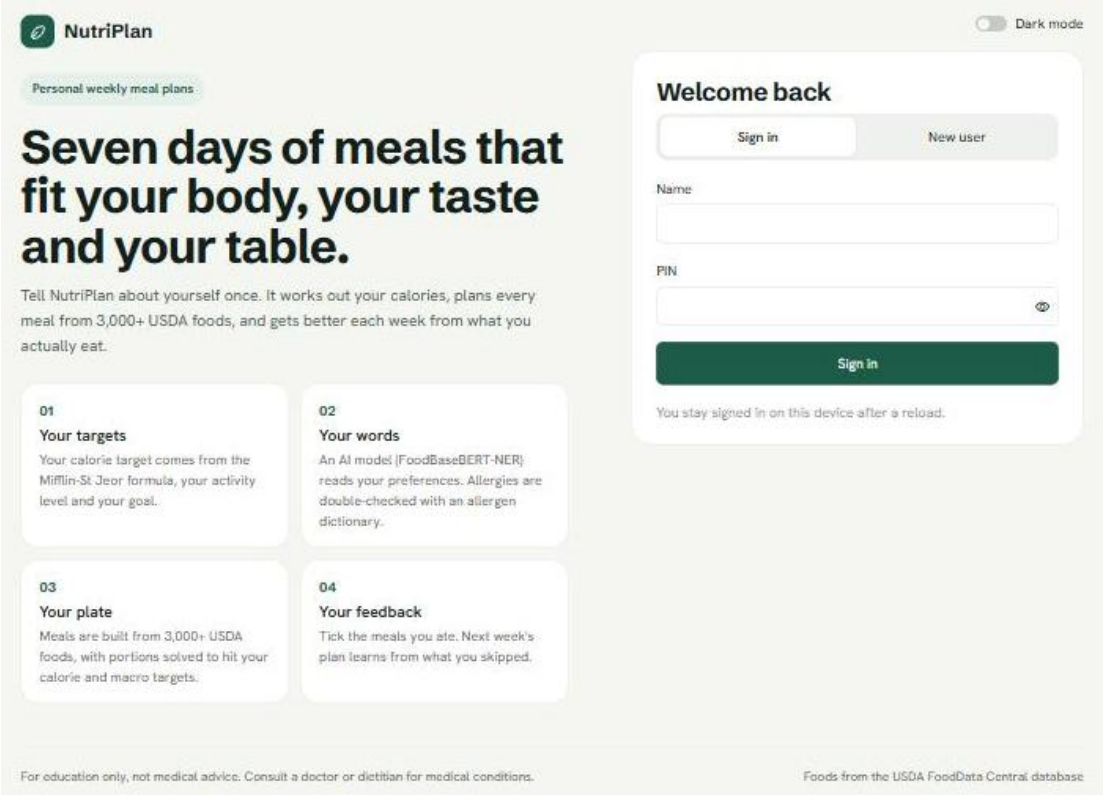


Figure 4-5 Welcome page with sign-in

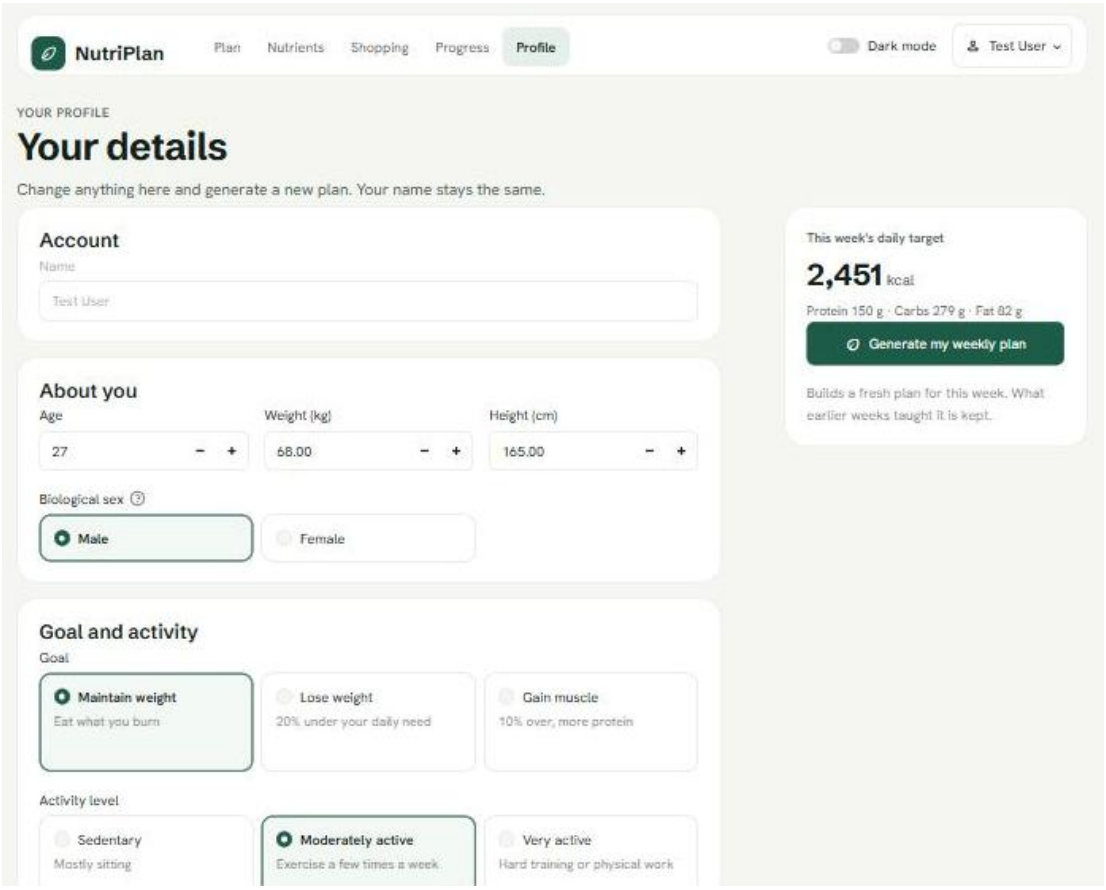


Figure 4-6 Profile page

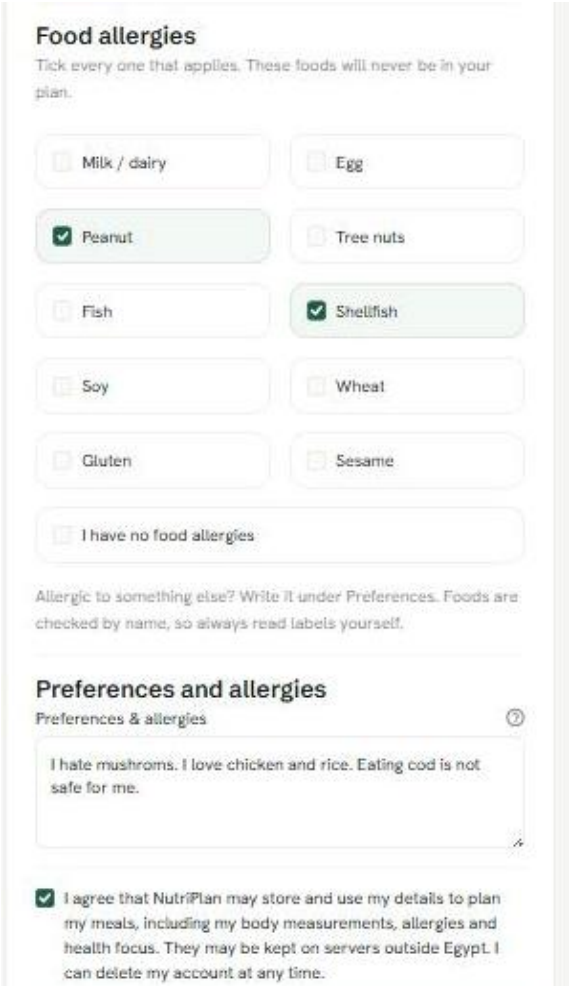


Figure 4-7 Allergy checklist, preferences and consent

The Plan page (Fig. 4-8) shows the daily targets and what the system understood. In this example, the typed text contains a spelling mistake, which is reported, and a sentence that the system could not act on, so it asks the user to check it.

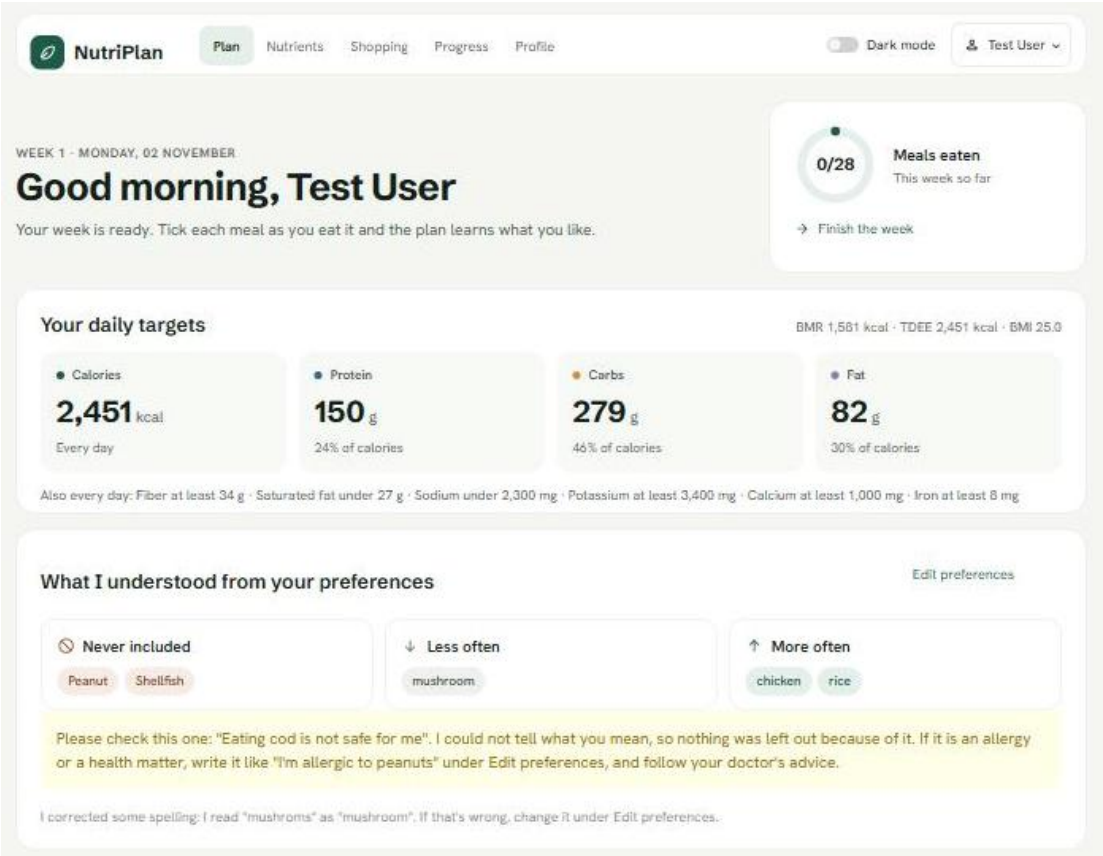


Figure 4-8 Plan page: targets, what was understood, and a request to check

Each meal card (Fig. 4-9) shows foods, grams and macronutrients, with buttons to swap a food, ask for a new meal, or mark it as eaten. Egyptian dishes also show their Arabic name and their ingredients. A new version appears beside the current one, and the application marks the one closer to the targets (Fig. 4-10). Each day ends with its totals (Fig. 4-11).

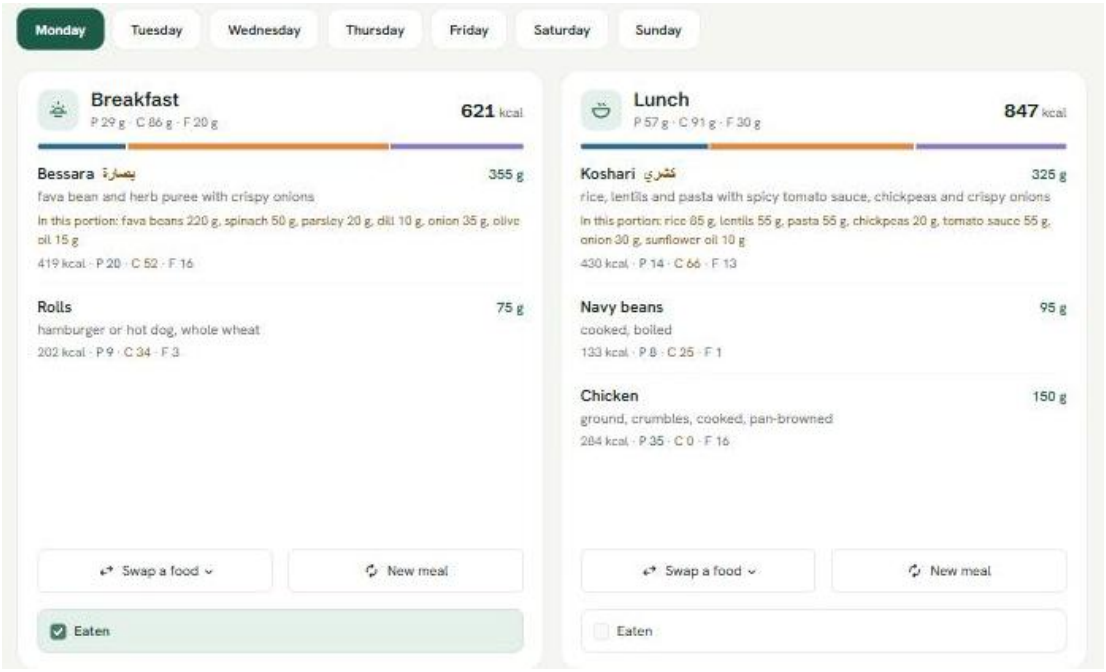


Figure 4-9 Meal cards

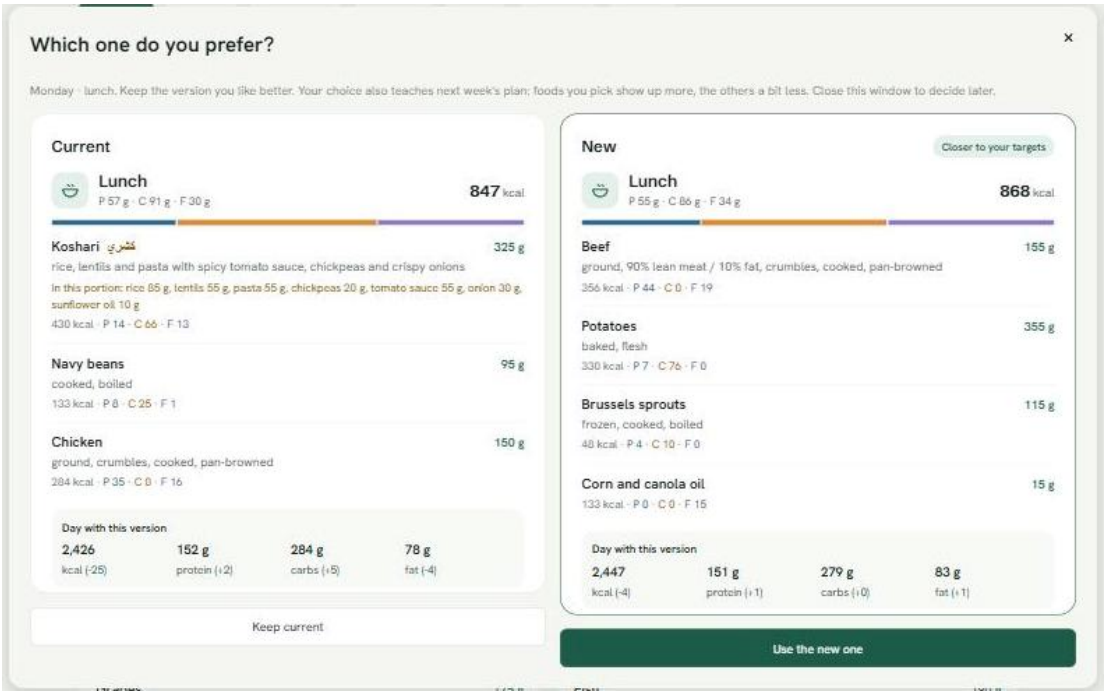


Figure 4-10 Comparing the current meal with a new version

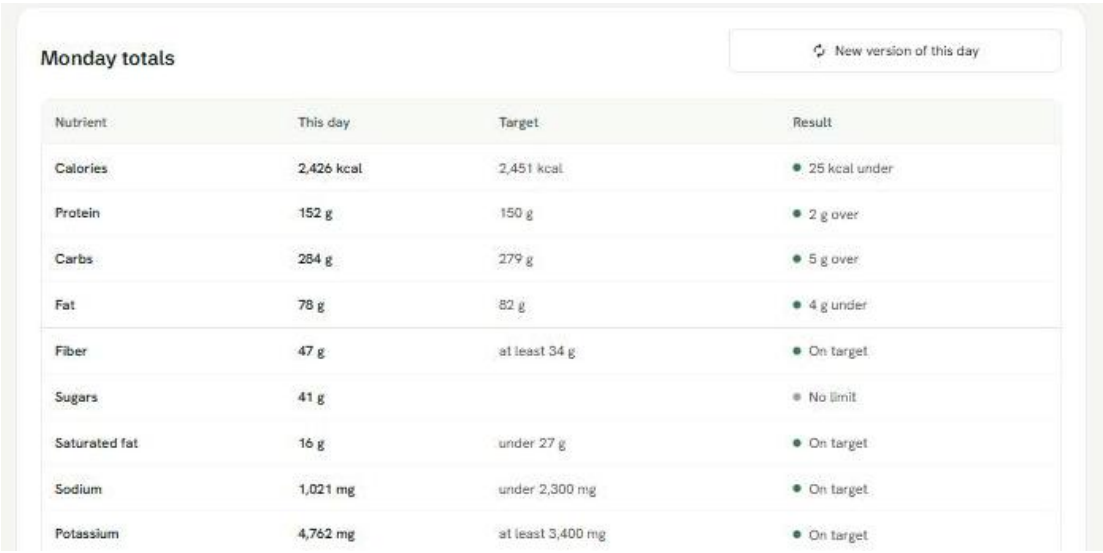


Figure 4-11 Day totals against targets

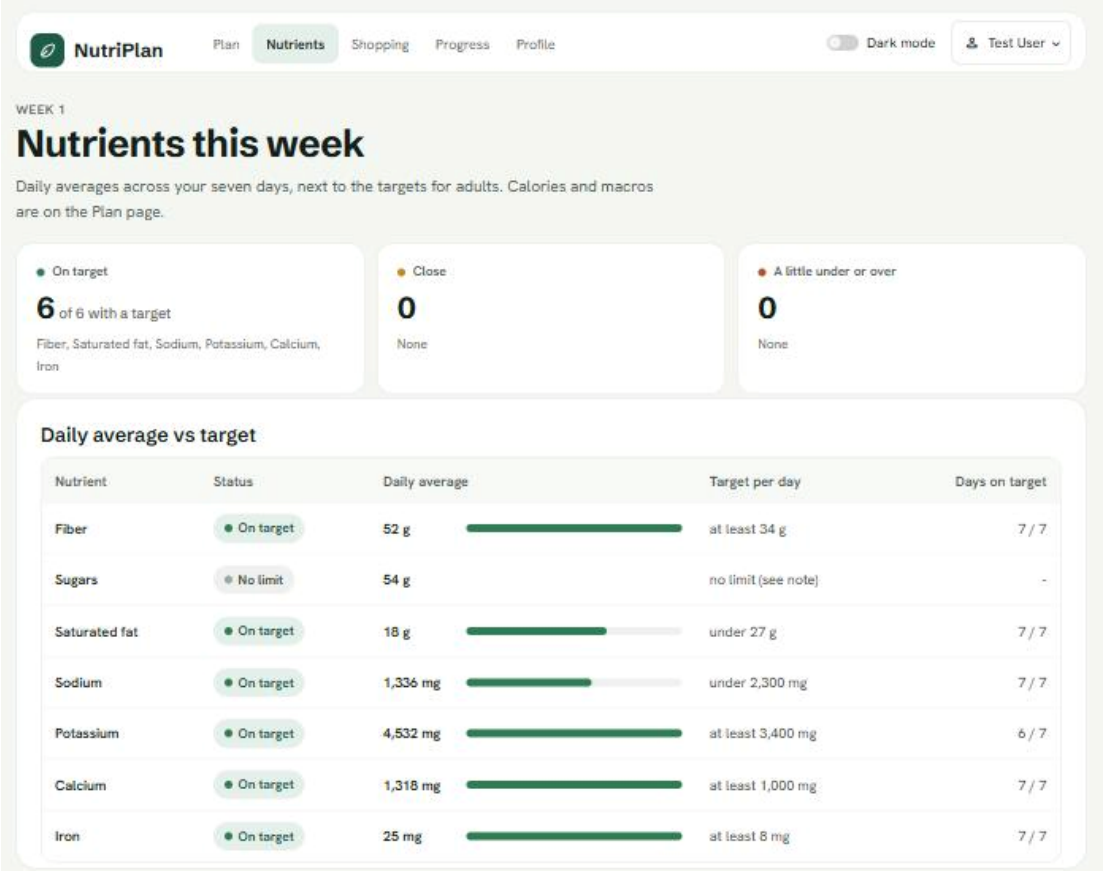


Figure 4-12 Nutrients page

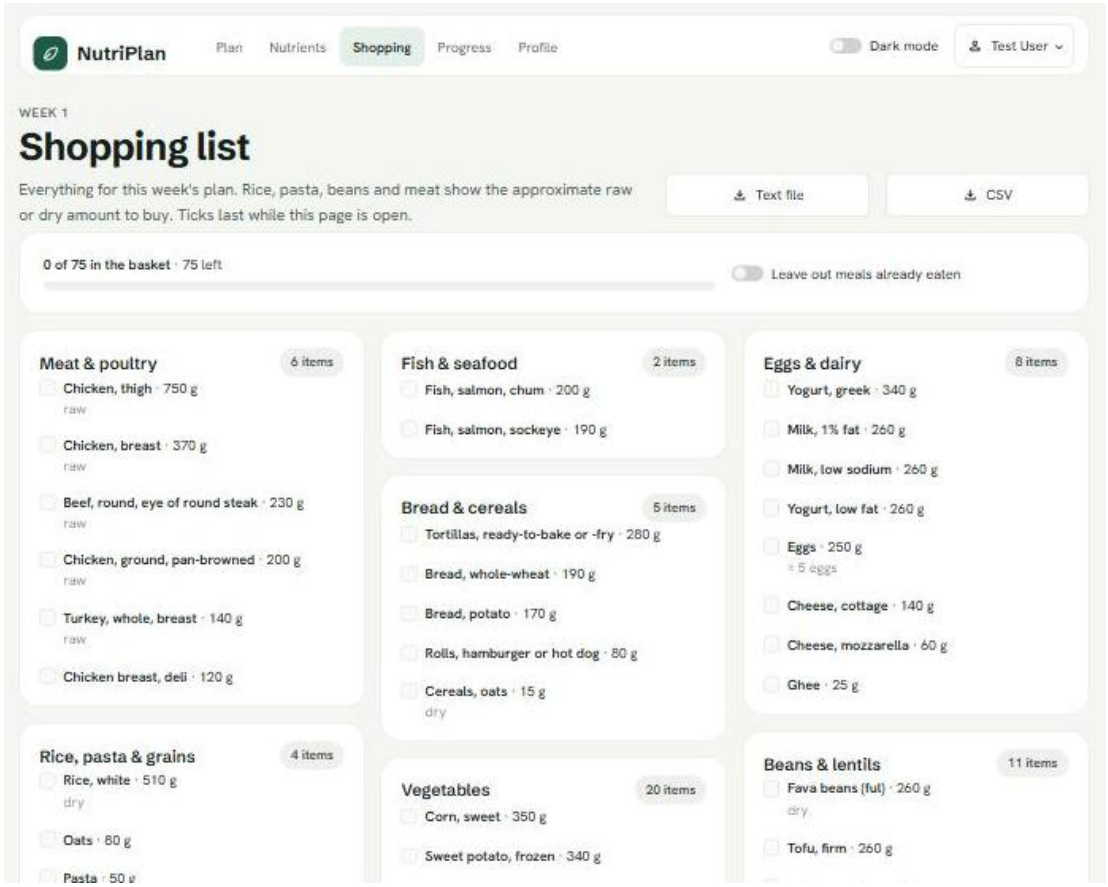


Figure 4-13 Shopping list, in raw or dry amounts to buy

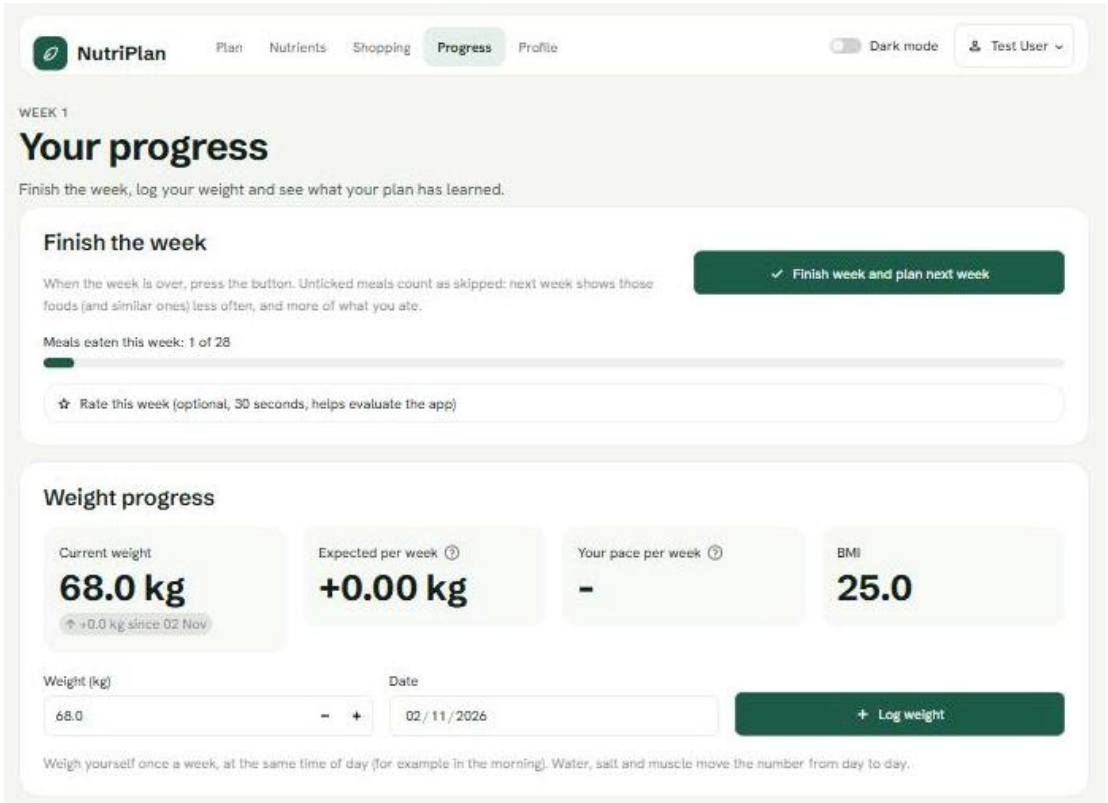


Figure 4-14 Progress page: finishing the week and tracking weight

4.2.4 Application Components

The full source code is available at https://github.com/mariofayez04/Grad-Project-Code, and the hosted application at https://grad-project-code.vercel.app. Table 4-3 lists the main components. They are joined together by plain function calls through app.py, and each person's data passes through db.py. Models load once and are shared between visitors, so a plan takes seconds after start-up. A plan stores its own targets and understood preferences, so a saved week does not change if a profile is edited later.

Table 4-3 Main components of the application

Component	Responsibility
app.py	Pages, forms, sign-in and dialogs
clean_data.py, allergens.py	Builds the food table; tags allergens and diets
calculator.py, nutrients.py	Targets, diet modes, nutrient reference values
parser.py	Reads free text into food rules
embeddings.py	Food vectors, computed once and saved
planner.py	Builds and re-plans meals; swap suggestions
feedback.py	Taste vector and penalties
dishes.py	Egyptian dishes from USDA ingredients
shopping.py, report.py, progress.py	Shopping list, PDF plan, weight progress
db.py	SQLite or PostgreSQL storage; PIN security
evaluate.py, testers.py, heldout.py	Evaluation tools

Three short pieces of code show the system's core. The function in Fig. 4-15 solves a meal's portions. Each row of the problem is one of the four targets, scaled so that errors are relative and calories count the most. SciPy finds the grams that come closest within each food's limits, and the result is rounded to 5 g, because nobody weighs 143.7 g of rice.

src/planner.py

```
323 def solve_grams(per100, lo, hi, target):
324     """Bounded least squares: grams of each food so the meal's kcal/protein/carbs/fat are close
            to target.
325     per100: n x 4 array (kcal, protein, carbs, fat per 100 g)."""
326     a = per100.T / 100.0                                        # 4 x n, per gram
327     b = np.array([target[m] for m in MACROS], dtype=float)
328     scale = np.array([2.0, 1.5, 1.0, 1.0]) / np.maximum(b, 1)   # relative error, calories
            matter most
329     res = lsq_linear(a * scale[:, None], b * scale, bounds=(lo, hi + 1e-9), method="bvls")  #
            bvls: ~10x faster for tiny problems
330     hi5 = np.maximum(np.floor(hi / 5) * 5, lo)                  # a maximum portion is a
            multiple of 5 g too
331     grams = np.clip(np.round(res.x / 5) * 5, lo, hi5)           # round to 5 g
332     error = float(np.sqrt(np.mean(((a @ grams - b) * scale) ** 2)))
333     return grams, error
```

Figure 4-15 Solving a meal's portions (src/planner.py)

The function in Fig. 4-16 is the learning step. It adds a share of the average

vector of eaten foods, subtracts a smaller share for skipped foods, and rescales the result. This is plain arithmetic on vectors from a pre-trained model, so it needs no training and no graphics card.

src/feedback.py

```
84 def update_taste(old, embeddings, eaten_ids, skipped_ids, alpha=ALPHA, beta=BETA):
85     """taste_new = normalize(taste_old + alpha * mean(eaten) - beta * mean(skipped))."""
86     v = np.zeros(embeddings.vectors.shape[1], dtype=np.float32) if old is None else
           np.asarray(old, dtype=np.float32).copy()
87     eaten_ids = [i for i in eaten_ids if i in embeddings.index]
88     skipped_ids = [i for i in skipped_ids if i in embeddings.index]
89     if eaten_ids:
90         v += alpha * embeddings.vectors[[embeddings.index[i] for i in eaten_ids]].mean(axis=0)
91     if skipped_ids:
92         v -= beta * embeddings.vectors[[embeddings.index[i] for i in skipped_ids]].mean(axis=0)
93     norm = np.linalg.norm(v)
94     return None if norm == 0 else v / norm
```

Figure 4-16 Updating the taste vector (src/feedback.py)

The function in Fig. 4-17 is part of the safety layer. Health wording near a food overrules a softer label, except that a weak word such as "stomach" does not overrule a stated like, so "yogurt is good for my stomach" stays a like.

src/parser.py

```
393 def health_upgrade(clause, s, e, label, cue):
394     """Health wording overrules a softer label. Blocking too much is safe; serving an allergen
            is not."""
395     if label == "block":
396         return label, cue
397     reaction = REACTION_RE.search(_part_around(clause, s, e)) or (label == "unknown" and
            REACTION_RE.search(clause))
398     if reaction and not (label == "prefer" and WEAK_REACTION_RE.match(reaction.group(0))):
399         return "block", reaction.group(0)
400     if label == "avoid" and MEDICAL_RE.search(clause):
401         return "block", MEDICAL_RE.search(clause).group(0)
402     return label, cue
```

Figure 4-17 Health wording overrules softer labels (src/parser.py)

Security. PINs are stored only as salted PBKDF2-SHA256 hashes with 200,000 rounds. Five wrong PINs lock an account for five minutes, and a wrong name and a wrong PIN give the same message. A page reload keeps the user signed in through a signed link, so typing another user's number in the address opens nothing. New users must agree to how their data is used before an account is created, and accounts made earlier are asked at their next sign-in. The Delete my account option asks for the PIN, then permanently removes the person's profile, plans, weights and ratings.

4.3 Testing

Testing methods were chosen to suit each risk, as Table 4-4 shows. The results are presented in Chapter 5.

Table 4-4 Testing methods

Method	What it checks	How
Unit tests	Formulas, parser, allergen tags, portions, learning	pytest, against hand-calculated values and hand-labeled cases
Integration tests	Complete flows through the interface	Streamlit's test tool: sign up, plan, swap, finish a week
Security tests	PIN handling and private links	Wrong PINs, locking, altered links, stored values
Database tests	SQLite and PostgreSQL give the same behavior	The whole test suite run on both databases
Parser tests	Wording the rules were not built on	New sentences, including a set scored only once
Simulated testers	Weeks of use, safety and learning	24 scripted users on a throwaway database
Deployment checks	The container and the hosted site	Local container with the host's limits; sign-in on the live site
Manual checks	Layout and interaction	Every page in a browser, at desktop and phone widths, in both themes

The parser tests were designed to reduce bias. Set 3 was written after the rules were frozen and scored only once. All three were written by an AI assistant, as were the simulated testers, so they are not a substitute for real users, as Chapter 5 discusses.

4.4 Summary

This chapter presented the layered, hybrid design of the system; the three data sources behind it; the working interface; the code at its core; and the testing methods. Fixed rules and calculation handle what must be dependable, and AI models handle reading text and comparing foods. Chapter 5 examines how well this worked.

Chapter 5
Results and Discussion

This chapter presents the results, tests what each part of the system adds, checks the results against the project's goals, and discusses further work and wider implications. One limit applies throughout: all users in these tests are simulated, and all new test sentences were written by an AI assistant, so the results show how the system behaves, not whether people would trust it.

5.1 Findings

The main product is a working web application, available on a PC and on a hosted site where sign-up and sign-in were confirmed by hand. All 281 automated tests pass, on both SQLite and PostgreSQL. Fig. 5-1 shows a run of the test suite.

```
============================= test session starts =============================
platform win32 -- Python 3.13.11, pytest-9.1.1, pluggy-1.6.0
collected 281 items
tests\test_allergens.py ....................................           [ 13%]
tests\test_app.py ........................                             [ 21%]
tests\test_calculator.py ..........................                    [ 30%]
tests\test_diet_modes.py ..............                                [ 34%]
tests\test_dishes.py ...........                                       [ 38%]
tests\test_embeddings.py .......                                       [ 41%]
tests\test_evaluation_tools.py ......                                  [ 43%]
tests\test_feedback.py ............                                    [ 47%]
tests\test_meals_per_day.py ...........                                [ 51%]
tests\test_nutrients.py ..............                                 [ 56%]
tests\test_parser.py ................................................. [ 74%]
...................                                                    [ 81%]
tests\test_planner.py ................                                 [ 87%]
tests\test_regenerate.py .........                                     [ 90%]
tests\test_report.py ....                                              [ 91%]
tests\test_shopping.py ................                                [ 97%]
tests\test_weight.py ........                                          [100%]
======================== 281 passed in 311.34s (0:05:11) ========================
```

Figure 5-1 A run of the automated test suite

5.1.1 Experimental Results

Accuracy. Table 5-1 shows how close planned days came to their targets for 30 random users over 7 days. Calories had a mean error of 2.3%, and 98.1% of days were within 10%. Protein was the least reliable at the extremes: 7.1% of days were more than 10% away. Weight-loss plans had the largest errors, probably because they fit similar nutrition into fewer calories.

Table 5-1 Planned days against targets (30 random users, 210 days)

Measure	Mean error	95th percentile	Days within 10%
Calories	2.3%	6.0%	98.1%
Protein	4.3%	13.9%	92.9%
Carbohydrate	3.0%	6.6%	99.5%
Fat	4.8%	10.2%	94.3%

Allergy safety when the allergy is understood. Across 20 random users with stated allergies, 0 of 1,652 food items contained a stated allergen, including 145 Egyptian dish servings. Across the 20 simulated testers using usual wording, 0 of 12,932 items broke a stated allergy or diet rule over four weeks. The allergen dictionary scored 100% precision and recall on 32 hand-labeled foods, although those cases were used while writing it, so that score is optimistic.

Allergy safety when the wording is new. This was the most important finding of the project. The parser scored 100% on the 32 sentences written during development, which suggested the problem was solved. New sentences showed otherwise: only 18 of 27 allergy items were blocked in the first set (Table 5-2). "im alergic to peanuts" blocked nothing, and four simulated testers with such wording were served 185 food items containing their stated allergen. A safety layer was added in response (Section 4.1). Those four testers then received none, although their exact sentences had helped to build the fix.

Table 5-2 Allergy items blocked by the parser, by set of sentences

Set	Sentences	Before the fix	After the fix	Not blocked, but the app asks
Set 1: used to build the fix	61	18 of 27 (66.7%)	27 of 27	0 of 0
Set 2: failures seen while fixing	40	7 of 30 (23.3%)	30 of 30	0 of 0
Set 3: kept apart, scored once	40	Not measured	22 of 32 (68.8%)	7 of 10

Set 3 is the fair measure, and Fig. 5-2 shows the pattern. The perfect scores on sets 1 and 2 show that the fix handles the wording it was built from, while 68.8% is closer to what new wording can expect. A later rule makes the application ask about unclear sentences, which covers 7 of the 10 misses, so 3 of 32 items (9.4%) were missed silently. That rule was added after seeing set 3, so it is not an independent result. One such sentence, "I blow up like a balloon after eating bread", even reads as a like, because of the word "like". Free text should therefore not be relied on for serious allergies.

Allergy checklist. This finding led to a checklist that needs no language model. Across 15 people who ticked allergen groups and typed nothing, 0 of 1,239 food items contained a ticked allergen, at a mean calorie error of 1.9%. If the people in set 3 also ticked the matching boxes, all 32 items would be blocked. That is true by design, not a test of behavior: it assumes that people tick the right box, which has not been tested with real users, and allergies outside the ten groups, such as the kiwi in set 3, still depend on free text.

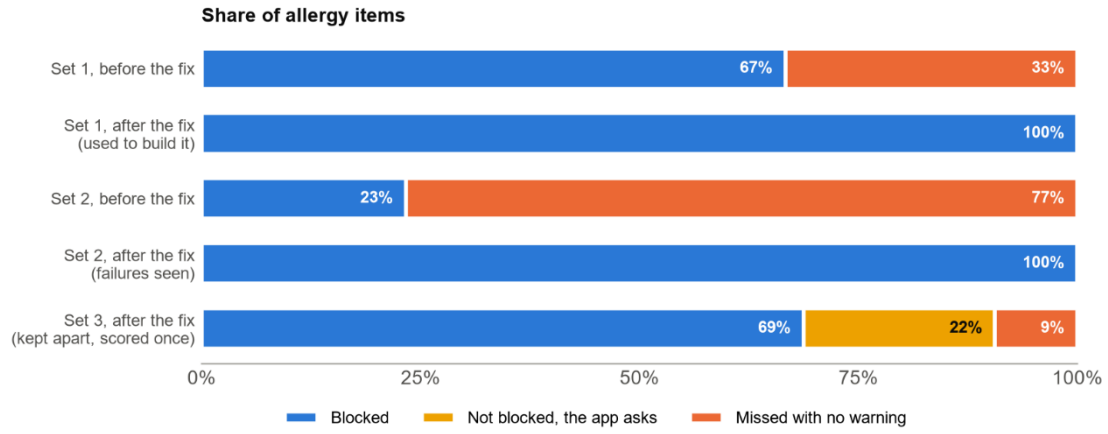


Figure 5-2 Allergy items handled by the parser, on each set of sentences

Likes and dislikes were weaker. On set 3, 100% of dislikes and 75% of likes were found, but on set 2 these were 50% and 25%. Indirect wording such as "Salmon is amazing" is often missed. The harm is smaller here, as a missed like only lowers the quality of the suggestions.

Learning. Fig. 5-3 shows 17 simulated testers with hidden dislikes that they never typed. With learning, the meals they ate rose from 78.4% in week 1 to between 87.4% and 90.6% after that, and disliked foods served fell from 5.8 to 0.8 a week. Without learning, both stayed about the same. These results depend on the assumed habits, such as skipping 90% of meals with a disliked food. They show that the mechanism works, not how quickly real tastes would be learned.

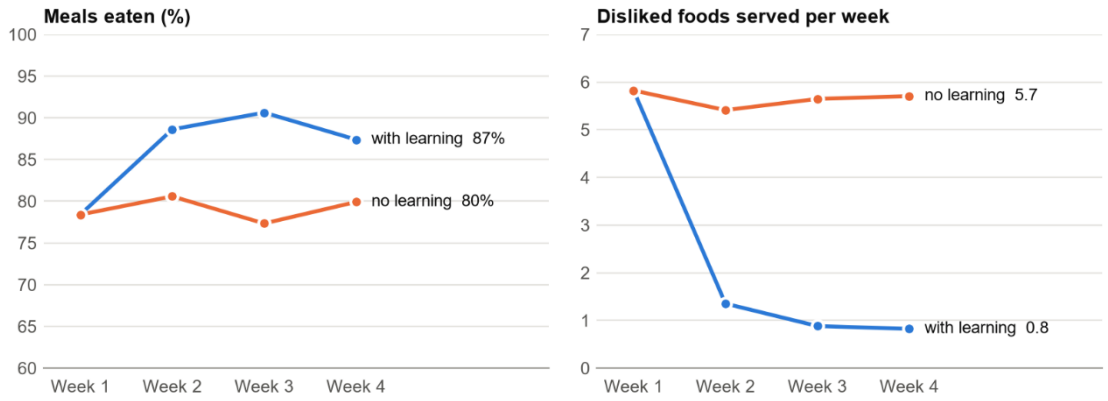


Figure 5-3 Simulated testers with and without weekly learning

Nutrients. Planning for nutrients as well as macronutrients improved every measure (Fig. 5-4). Days over the sodium limit fell from 46.4% to 0%. This cost a little accuracy: macronutrient error rose from 3.1% to 4.2% on the same users, which was accepted. Calcium remains the hardest, on target on 82.1% of days. In diet modes, 8 random users received 530 mg of sodium a day in low-salt mode, and 32 g of sugars in diabetes-friendly mode, with no excluded foods served. These are planned amounts, which do not count salt added in cooking, and they are not evidence of any health

outcome.

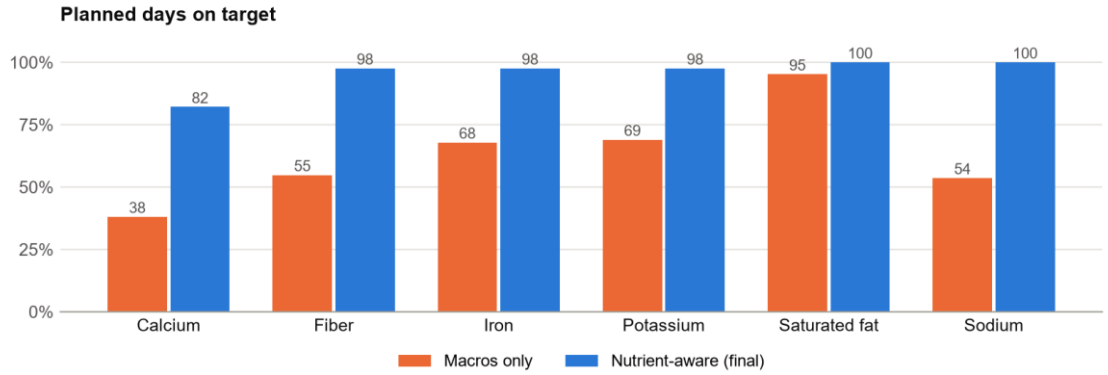


Figure 5-4 Days on target, with and without nutrient-aware planning (84 days)

Other results. A week took 3.42 seconds on average to plan (4.14 seconds at most) on the laptop. Egyptian dishes made up about 6.2 meals a week on the "sometimes" setting and 10.5 on "often". On the hosted site, a visitor can wait about 20 seconds after an idle period, based on a container test, and the free plan closes the live connection every 5 minutes.

Real use. The local database held 4 profiles and 10 plans, with only 1 week finished, by 1 person, who ticked 89.3% of meals. No ratings were given. This is too little to draw any conclusion, and user satisfaction remains unmeasured.

Unexpected results. Fortified cereals gave some days over 60 mg of iron, above the safe limit, before daily caps were added. The spelling correction first read "chickpeas" as "chicken", which could have blocked the wrong food, until all food names were excluded from correction. Hosted logs also showed eight server starts within about a minute of one page load, which could use up a free allowance faster than expected.

5.1.2 Ablation Study

Each part was removed in turn, or compared with a simpler version, to show what it adds (Table 5-3). The most surprising result concerns the NER model. Allergy recall was nearly the same with the model alone or the dictionary alone, because most allergens are common words that both find. The model's value is finding foods outside the dictionary, which raises recall of "never" items from 94.3% to 100% on set 1, and from 66.7% to 69.7% on set 3. Allergy failures came from the wording around the food rather than from finding the food. A better food model would therefore not fix them.

Table 5-3 Ablation study: what each part adds

Part removed or changed	Measure	Without it	With it
NER model (dictionary only)	"Never" items found, set 3	66.7%	69.7%
Dictionary (NER model only)	Allergy items blocked, set 3	65.6%	68.8%
Safety layer	Allergy items blocked, set 2	23.3%	100% (seen)
Weekly learning	Meals eaten, week 3	77.3%	90.6%

Part removed or changed	Measure	Without it	With it
Nutrient-aware planning	Days on calcium target	38.1%	82.1%
Nutrient-aware planning	Macronutrient error	3.1%	4.2% (a cost)
Category in embedded text	Same category in 5 neighbors	88.4%	98.1%

5.2 Goals Achieved

Table 5-4 compares the results with the objectives from Section 1.3. Five were achieved and two only partly, including the one that matters most for safety. Beyond the plan, the project also added private accounts, diet modes, Egyptian dishes, weight tracking, printable plans and shopping lists.

Table 5-4 Objectives against results

Objective	Result	Evidence
1. Calculate targets	Achieved	Tests against hand-calculated values
2. Prepare clean food data	Achieved	3,204 foods with allergen and diet tags
3. Read preferences and apply allergies	Partly	Ticked allergies never served; free text blocked 68.8% of new-wording items
4. Generate a week with portions	Achieved	2.3% calorie error; 98.1% of days within 10%
5. Learn from behavior	Achieved in simulation	Disliked foods 5.8 to 0.8 a week; no real-user test
6. Simple interface, private accounts	Achieved	Tests and manual checks; usability not tested with people
7. Evaluate and state limits	Partly	Measured, but no real-user study

The non-functional requirements in Table 3-2 were mostly met, as measured: calorie accuracy (NFR1), speed on the laptop (NFR3), PIN security (NFR4), privacy of names and links (NFR5), portability (NFR8), free tools (NFR9) and automated tests (NFR10). Reliability (NFR7) was shown by restarting the app and database in a test environment, and by signing in again on the live site. Safety (NFR2) was met for ticked and understood allergies, but free text alone remains unreliable, and usability (NFR6) was checked by the developer alone.

5.3 Further Work

5.3.1 Enhancements and Future Features

- Understanding allergy sentences by meaning, for instance a sentence classifier trained on health statements, since failures came from the wording around foods.
- Extending the allergy checklist beyond its ten groups, and testing with real people whether they complete it correctly.
- Support for Arabic preferences, which suits the Egyptian users this project aims at.
- More Egyptian dishes, with nutrients checked against measured data, and more food data with ingredient lists.
- A mobile application, and a hosting plan that avoids the 5-minute disconnection.

5.3.2 Limitations and Incomplete Work

- No real-user study: the rating form and volunteer sheet exist, but were not used. Satisfaction, usability and real acceptance are unmeasured. This is the largest gap, caused by time.
- All new test sentences came from an AI assistant, which knew the system. Real people may write differently.
- Allergens are found from food names, so an ingredient not named cannot be detected. Food names from the USDA are American, and estimates fill some nutrient gaps.
- The nutrient values follow the 2020-2025 US guidelines, and a 2025-2030 edition has since been published. They are not Egyptian guidance.
- The hosted site was not tested with many users at once.

5.4 Social and Ethical Implications

Social. The project could help people eat better at no cost, and the inclusion of Egyptian dishes makes it more relevant to local users than many imported tools. It could also do harm. A weight-focused tool can be risky for people with eating disorders, so the application includes a calorie floor, warnings on low body weight and on losing weight too fast, and a note to follow medical advice. It does not detect eating disorders, and people with them are outside its scope.

Professional and safety. Nutrition advice affects health, so the application states that it is for education only and is not medical advice, and diet modes state that they are not a treatment. The largest professional concern is a missed allergy, which is why a checklist was added and why food names cannot replace reading labels. For the same reason, this report gives the less flattering 68.8% as the fair figure instead of the 100% from the sentences that shaped the rules, and it says plainly which results are simulated.

Legal and privacy. Body measurements and health focus are personal data. Egypt's personal data protection law (Arab Republic of Egypt, 2020) treats health data as sensitive, and summaries of the law describe explicit consent as the general rule for processing it. Reviewing this during the project led to two additions: a required consent box when signing up, which also states that data may be kept outside Egypt, as the hosted version does, and a Delete my account option that removes all of a person's data from the live database. The application also limits what it collects, stores PINs only as hashes, keeps data out of its code repository, and sends no data to outside AI services. This was not reviewed by a lawyer, and it should be before any public use beyond testing. The data and models are used under open terms: USDA data is public, FoodBaseBERT-NER is under the MIT license, and all-MiniLM-L6-v2 is under Apache 2.0.

Fairness and transparency. The energy equation has limits for individuals (Frankenfield et al., 2005), and the form asks only for biological sex because the equation requires it. The system explains what it understood, and lets users change it.

AI tools were used to help build this project and to write its test sentences and simulated testers, which is stated wherever it affects a result.

5.5 Summary

The system met its targets for calories and nutrients, kept stated allergens out of plans when it understood them, and learned from simulated behavior. Testing on new wording found a serious weakness that the development tests had hidden: even after a fix, 68.8% of allergy items were blocked on a fair test, which led to an allergy checklist that needs no language model. The main gap is the lack of a study with real users.

Chapter 6
Conclusions

This project set out to make personal meal planning easier by building a system that plans a week of meals around a person's body, tastes and allergies, and improves from what they actually eat. This chapter summarizes the work and its main contributions.

6.1 Summary of the Work

The project began from a common problem: general diet plans ignore individual differences, and working out a personal plan takes knowledge and time. A review of existing applications and research showed that commercial tools lean towards tracking or paid coaching, that research systems tend to address one part of the problem, and that handling allergies safely is rarely studied.

NutriPlan AI was then built as a working web application. The USDA SR Legacy data was cleaned to 3,184 foods with allergen and diet tags, and 20 Egyptian dishes were added. Targets are calculated with the Mifflin-St Jeor equation. Free text is read by a food NER model with a dictionary and rules. Meals are assembled from suitable foods, with portions solved by bounded least squares. Each week's plan learns from the last through a taste vector built on pre-trained embeddings, and the application also offers private accounts, swaps, diet modes, shopping lists, PDF plans and weight tracking. It runs on a PC and on free hosting, with accounts kept in a permanent database.

Testing showed that plans meet their targets closely, with a mean calorie error of 2.3% and 98.1% of days within 10%. No stated allergen reached a plan when the allergy had been understood, and simulated users with hidden dislikes received far fewer of those foods with learning than without. The most valuable result was a negative one. A parser that scored 100% on its own sentences blocked only 18 of 27 allergy items on new wording. After a safety layer was added, it blocked 22 of 32 on a set kept apart from the work, so an allergy checklist was added that needs no language model at all.

The project is not finished as a public service. It has not been tested with real people, so user satisfaction is unknown. It needs a legal review before public use, and a better way of understanding allergy sentences. Even so, the objectives were met in full or in part, and the system shows that useful personal planning is possible with free data, free models and modest hardware.

6.2 Main Contributions

The main contributions of the project are:

1. **A complete, working system.** It joins free-text preferences, fixed allergy rules,

calculated portions, nutrient-aware planning and weekly learning in one application, rather than studying one of them alone.

2. **A division of work between AI and rules.** Models read text and compare foods, while formulas, optimization and a fixed dictionary handle numbers and safety. The ablation study supports this: allergy failures came from wording, not from finding foods.
3. **Evidence on testing language-based safety features.** The gap between 100% on development sentences and 68.8% on sentences kept apart shows why a food-safety feature must be tested on wording it has not seen, and why it should ask when unsure, or avoid free text for the most serious cases.
4. **Meal planning with Egyptian food.** Twenty dishes are built from USDA ingredients, with calculated nutrients and allergens, so that they take part in planning, swaps, shopping lists and learning like any other food.
5. **A reusable evaluation approach.** Simulated testers, a comparison with learning switched off, and tools ready for a real-user pilot allow later work to repeat and extend these results.

Above all, the project showed that being honest about a system's limits is part of building it safely. The next step is the one this project could not take: putting it in front of real people and measuring what happens.

References

Adomavicius, G. and Tuzhilin, A. (2005) 'Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions', IEEE Transactions on Knowledge and Data Engineering, 17(6), pp. 734–749.

Afshin, A. et al. (2019) 'Health effects of dietary risks in 195 countries, 1990–2017: a systematic analysis for the Global Burden of Disease Study 2017', The Lancet, 393(10184), pp. 1958–1972.

Appel, L.J. et al. (1997) 'A clinical trial of the effects of dietary patterns on blood pressure', New England Journal of Medicine, 336(16), pp. 1117–1124.

Arab Republic of Egypt (2020) Law No. 151 of 2020 Promulgating the Personal Data Protection Law. Cairo.

Burke, R. (2002) 'Hybrid recommender systems: survey and experiments', User Modeling and User-Adapted Interaction, 12(4), pp. 331–370.

Cronometer (2026) Cronometer. Available at: https://cronometer.com/ (Accessed: 2 November 2026).

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics, pp. 4171–4186.

Dizex (no date) FoodBaseBERT-NER. Available at: https://huggingface.co/Dizex/FoodBaseBERT-NER (Accessed: 2 November 2026).

Eat This Much (2026) Automatic Meal Planner. Available at: https://www.eatthismuch.com/ (Accessed: 2 November 2026).

Elsweiler, D. and Harvey, M. (2015) 'Towards automatic meal plan recommendations for balanced nutrition', Proceedings of the 9th ACM Conference on Recommender Systems, pp. 313–316.

Frankenfield, D., Roth-Yousey, L. and Compher, C. (2005) 'Comparison of predictive equations for resting metabolic rate in healthy nonobese and obese adults: a systematic review', Journal of the American Dietetic Association, 105(5), pp. 775–789.

Hall, K.D. (2008) 'What is the required energy deficit per unit weight loss?', International Journal of Obesity, 32(3), pp. 573–576.

Institute of Medicine (2001) Dietary Reference Intakes for Vitamin A, Vitamin K, Arsenic, Boron, Chromium, Copper, Iodine, Iron, Manganese, Molybdenum, Nickel, Silicon, Vanadium, and Zinc. Washington, DC: National Academies Press.

Institute of Medicine (2011) Dietary Reference Intakes for Calcium and Vitamin D. Washington, DC: National Academies Press.

Khamesian, S., Arefeen, A., Carpenter, S.M. and Ghasemzadeh, H. (2025) 'NutriGen: personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence', arXiv preprint, arXiv:2502.20601.

Lops, P., de Gemmis, M. and Semeraro, G. (2011) 'Content-based recommender systems: state of the art and trends', in Ricci, F., Rokach, L., Shapira, B. and Kantor, P.B. (eds.) Recommender Systems Handbook. New York: Springer, pp. 73–105.

Mifflin, M.D. et al. (1990) 'A new predictive equation for resting energy expenditure in healthy individuals', The American Journal of Clinical Nutrition, 51(2), pp. 241–247.

MyFitnessPal (2026) MyFitnessPal. Available at: https://www.myfitnesspal.com/ (Accessed: 2 November 2026).

National Academies of Sciences, Engineering, and Medicine (2019) Dietary Reference Intakes for Sodium and Potassium. Washington, DC: National Academies Press.

Niszczota, P. and Rybicka, I. (2023) 'The credibility of dietary advice formulated by ChatGPT: robo-diets for people with food allergies', Nutrition, 112, 112076.

Noom (2026) Noom: Weight Loss and GLP-1 Support [Mobile app listing]. Available at: https://apps.apple.com/us/app/noom-weight-loss-glp-1-support/id634598719 (Accessed: 2 November 2026).

Popovski, G., Seljak, B.K. and Eftimov, T. (2019) 'FoodBase corpus: a new resource of annotated food entities', Database, 2019, baz121.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: sentence embeddings using Siamese BERT-networks', Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 3980–3990.

Stark, P.B. and Parker, R.L. (1995) 'Bounded-variable least-squares: an algorithm and applications', Computational Statistics, 10, pp. 129–141.

Stigler, G.J. (1945) 'The cost of subsistence', Journal of Farm Economics, 27(2), pp. 303–314.

Stojanov, R., Popovski, G., Cenikj, G., Koroušić Seljak, B. and Eftimov, T. (2021) 'A fine-tuned bidirectional encoder representations from transformers model for food named-entity recognition: algorithm development and validation', Journal of Medical Internet Research, 23(8), e28229.

Tran, T.N.T., Atas, M., Felfernig, A. and Stettinger, M. (2018) 'An overview of recommender systems in the healthy food domain', Journal of Intelligent Information Systems, 50(3), pp. 501–526.

Trattner, C. and Elsweiler, D. (2017) 'Food recommender systems: important contributions, challenges and future research directions', arXiv preprint, arXiv:1711.02760.

U.S. Department of Agriculture, Agricultural Research Service (2018) FoodData Central: SR Legacy, April 2018 release. Available at: https://fdc.nal.usda.gov/download-datasets (Accessed: 2 November 2026).

U.S. Department of Agriculture and U.S. Department of Health and Human Services (2020) Dietary Guidelines for Americans, 2020–2025. 9th edn. Washington, DC: U.S. Government.

Virtanen, P. et al. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', Nature Methods, 17(3), pp. 261–272.

World Health Organization (2026) Healthy Diet. Available at: https://www.who.int/news-room/fact-sheets/detail/healthy-diet (Accessed: 2 November 2026).

Yang, L. et al. (2017) 'Yum-me: a personalized nutrient-based meal recommender system', ACM Transactions on Information Systems, 36(1), pp. 1–31.

الملخص العربي

يصعب على كثير من الناس الالتزام بنظام غذائي صحي، لأن الخطط الغذائية الجاهزة لا تراعي الفروق بين الأفراد، ولأن إعداد خطة شخصية يتطلب معرفة ووقتًا لا يتوفران للجميع. ومن هنا جاءت فكرة هذا المشروع، الذي يهدف إلى بناء نظام ذكي يقترح وجبات أسبوع كامل تناسب جسم المستخدم وذوقه وحالته الصحية، ثم يتحسن مع الوقت بناءً على ما يأكله فعلًا.

نتج عن المشروع تطبيق ويب مجاني يعمل بالفعل باسم NutriPlan AI. يعتمد التطبيق على بيانات وزارة الزراعة الأمريكية بعد تنقيحها إلى 3,184 صنفًا غذائيًا، أضيفت إليها 20 أكلة مصرية مثل الكشري والملوخية والفول المدمس. ويحسب النظام احتياجات المستخدم من السعرات والعناصر الغذائية بمعادلات علمية معتمدة، ويقرأ تفضيلاته وحساسيته الغذائية من نص يكتبه بأسلوبه الخاص، مستخدمًا نموذج ذكاء اصطناعي مدعومًا بقواعد ثابتة للسلامة. أما كميات الطعام فتُحسب بطريقة رياضية لا تترك للنموذج، كما يتعلم النظام كل أسبوع من الوجبات التي تناولها المستخدم أو تركها.

أظهرت الاختبارات أن متوسط الخطأ في السعرات بلغ 2.3%، وأن 98.1% من الأيام جاءت في حدود 10% من الهدف، ولم يصل أي مسبب حساسية إلى الخطة متى فهمه النظام. غير أن أهم نتيجة جاءت من اختبار جمل جديدة لم يُبنَ عليها النظام، إذ كشفت ضعفًا لم تظهره الاختبارات الأولى. وبعد إضافة طبقة للسلامة، منع النظام 68.8% من حالات الحساسية في مجموعة جمل حُفظت جانبًا لهذا الغرض، وصار يسأل المستخدم عن معظم الحالات الأخرى. ولهذا لا يصح الاعتماد على النص وحده في حالات الحساسية الخطيرة، فأضيفت قائمة اختيارات لمسببات الحساسية لا تعتمد على أي نموذج لغوي.

وأجريت الاختبارات على مستخدمين افتراضيين، ولم تكتمل تجربة مع مستخدمين حقيقيين، فبقي رضا المستخدمين غير مقيس، وهذه هي الخطوة التالية الأهم. كما أضيفت خلال المشروع موافقة صريحة على حفظ البيانات وإمكانية حذف الحساب، مراعاةً لخصوصية البيانات الصحية. ويبين المشروع في مجمله أن التخطيط الغذائي الشخصي ممكن ببيانات ونماذج مجانية، بشرط توزيع الأدوار بين الذكاء الاصطناعي والقواعد الثابتة، والصدق في عرض حدود النظام.